Malware Analysis - Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Claude to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

CHAPTER 1: Introduction

Malware Analysis and Incident Forensics

The Anatomy of an Intrusion: The Cyber Kill Chain

Every cyberattack, however sophisticated or crude, can be decomposed into a sequence of discrete stages. This decomposition is not merely academic: it gives defenders a shared vocabulary for describing where in an attack's lifecycle a particular piece of evidence belongs, and it gives malware analysts a checklist of behaviors to expect and hunt for when dissecting a sample. The most widely adopted model for this decomposition is the **intrusion kill chain**, originally formulated by Lockheed Martin. It describes, stage by stage, the process by which perpetrators carry out a cyberattack, from the earliest scouting activities through to the final exploitation of the compromised environment.

The chain begins with **Reconnaissance**, the stage in which the attacker studies the target in search of vulnerabilities. This search is not limited to software flaws: it extends to the identification of weak points in an organization's technical stack (outdated software, misconfigured services) as well as its human element (employees who might be susceptible to social engineering). Reconnaissance is the intelligence-gathering phase that determines what avenues of attack will even be feasible.

Once a vulnerability or weakness has been identified, the attacker moves to **Weaponization**. This is the stage in which the attacker crafts an artifact capable

of exploiting the discovered vulnerability — for instance, engineering a payload that triggers a buffer overflow in a specific piece of software. Weaponization transforms abstract knowledge of a flaw into a concrete, usable exploit.

The third stage is **Delivery**, the mechanism by which the weaponized artifact is transported to the victim. Delivery can take many forms: a phishing email carrying a malicious attachment, a booby-trapped USB drive left in a parking lot, a drive-by download from a compromised website, and so forth. The delivery mechanism is often what a defender first observes, since it is the point at which the malicious content crosses into the victim's environment.

Exploitation follows delivery: this is the moment at which the vulnerability identified during reconnaissance and targeted during weaponization is actually triggered, granting the attacker some initial foothold or code execution capability on the target system.

After exploitation succeeds, the attacker typically seeks to secure and extend that foothold. This is where the behavioral properties that malware analysts are trained to recognize come into play.

Persistence

Persistence is the property by which malware achieves and guarantees its own survivability within the compromised system over time. Consider a straightforward but illustrative scenario: the infected machine is rebooted, meaning the computer is fully shut down and then powered back on. If persistence has been correctly achieved, the malware will resume execution automatically the moment the system boots again, without any further action required from the attacker. In other words, persistence is about ensuring the malicious code survives beyond the singular moment of infection.

To achieve this survivability, malware typically must tamper with the operating system's configuration — modifying registry keys, scheduled tasks, service definitions, startup folders, library search paths, or similar mechanisms that the OS consults when deciding what code to execute automatically. This tampering process can vary enormously in character: it might be conducted in a very noisy, conspicuous fashion, or it might be executed with extreme care to avoid detection. Regardless of the specific technique, the changes made to achieve persistence are frequently designed to be hidden from casual inspection, since an administrator who stumbles upon an unfamiliar autorun entry might well investigate and remove it.

Uniqueness ("Single-Instance" Behavior)

A second behavioral property commonly exhibited by malware is **uniqueness**, sometimes described as "single-instance" execution. Malware authors generally want to guarantee that only one copy of their malicious code is running on a given machine at any time. This might seem counterintuitive at first glance — surely more running instances of malicious code would only benefit the attacker? — but in practice, multiple concurrent executions of the same malware are frequently meaningless or even actively harmful to the malware's own operation. Two instances might compete for the same system resources, interfere with each other's tasks, or produce undesired side effects. A vivid illustration of this danger is ransomware: if two instances of the same ransomware strain began encrypting the filesystem simultaneously, the result could be file corruption or an encryption process that breaks its own functionality, rendering the ransom note's promise of a decryption key useless even to the attacker. To prevent this outcome, malware commonly performs start-up checks — for instance, creating a named mutex or checking for a marker file — to verify whether the machine has already been infected. If an existing infection is detected, the new instance will typically decline to create a second infection, though it might instead choose to update the existing one.

Targeting

Not all malware is indiscriminate. Some threats are explicitly designed to infect only specific, pre-selected victims, and such malware needs a way to recognize whether it has landed on the intended target or on some other, irrelevant machine. This behavioral property is called **Targeting**. When a sample lands on a machine that is not the intended target, it might choose to terminate its own execution quietly, or it might instead disguise itself by behaving like an entirely different, more innocuous piece of malware or software, thereby throwing off analysts who happen to be observing it.

The mechanisms available to malware for recognizing the "right" victim machine are numerous. A classic technique, especially prevalent among banking trojans, is checking regional and language settings — a piece of malware targeting Eastern European banks, for example, might terminate immediately if it detects that the system locale is set to English (United States). Another approach is checking hardware and software characteristics of the machine, such as the presence of a specific CPU model, particular installed software, or particular configuration settings. Malware can also check for the presence of specific external devices, or

search for known documents that would only be present on a machine belonging to the intended target organization. In essence, the malware fingerprints the machine it has landed on in order to determine whether it matches the profile of the intended victim.

Obfuscation and Evasion

Eventually, defenders will notice something is wrong, and analysts will begin dissecting the malware to understand how it operates. Malware authors know this is inevitable and therefore invest considerable effort into making their creations difficult to analyze. This concern gives rise to two closely related properties: **Obfuscation** and **Evasion**.

Obfuscation refers to techniques that make data and code structures genuinely difficult to interpret, even when an analyst has full access to the binary. This can involve scrambling the malware's own code and data structures using complex, deliberately convoluted schemes, so that any easily discoverable hints of the sample's malicious nature are hidden from casual static inspection. Malware may also inject its code into the address space of other legitimate processes in order to conceal its real identity from process-monitoring tools, or it may employ a broader arsenal of techniques specifically engineered to hamper the analyst's ability to correctly understand its behavior. Malicious code might also be spread across multiple components or delivery stages, making it harder for an analyst to assemble the complete picture of how the attack functions, since no single artifact contains the whole story.

Evasion, meanwhile, refers to the malware's ability to actively hide its own presence and effects while operating, and to present a different, innocuous appearance if it detects that it is being analyzed — for instance, by a sandbox, a debugger, or antivirus software. The overarching goal of both obfuscation and evasion is to allow the malware to remain undercover on the infected machine for as long as possible, maximizing the window during which it can operate undetected. There are, however, notable exceptions to this "stay hidden" philosophy: ransomware, for example, has no interest whatsoever in remaining covert once it has finished encrypting the victim's files, since its entire business model depends on making its presence loudly and unmistakably known in order to extract a ransom payment.

Fingerprinting and Beaconing

Once the attack has succeeded in compromising the machine, the malicious software typically needs to inform the attacker that the operation was successful. This is accomplished by reusing many of the same techniques employed for targeting: the malware inspects the infected system and collects descriptive information about it, then transmits this description back to the attacker, commonly via a remote server under the attacker's control. This notification stage is referred to as **Beaconing** — the malware "beacons" its presence and characteristics back to its controller, confirming that the infection has taken hold and providing the attacker with the details needed to manage the newly compromised asset.

Communication

Beyond the initial beacon, malware frequently maintains ongoing **Communication** with external Command and Control (C2) infrastructure. This communication channel serves multiple purposes: it allows the malware to receive further instructions and commands from the attacker, to update its own internal components (effectively self-updating in the field), and to exfiltrate data harvested from the victim machine back to the attacker.

The methodologies used to implement this communication are impressively heterogeneous, reflecting the attacker's need to blend malicious traffic in with the ordinary noise of network activity. Communication may take place through direct links to attacker-controlled servers, but it can equally be routed through online storage services, from which the malware simply reads commands as though it were retrieving an ordinary file — from a defender's vantage point, all that is visible is a request to a legitimate cloud storage provider, with nothing to obviously flag it as malicious. Social media platforms present another channel: fake accounts are sometimes used to host commands or facilitate data exfiltration, again blending in with ordinary internet traffic. Malware may also communicate through network services, most notably DNS queries, using the domain name resolution system as a covert channel for encoding and transmitting information. Whatever channel is chosen, the data exchanged over it is frequently obfuscated, ensuring that even if the traffic itself is intercepted, its contents remain opaque to an observer.

CHAPTER 2: x86 Assembly

The Nature of Binary Analysis

The vast majority of the malware samples an analyst will encounter arrive in the form of executables — binary content that a processor can directly execute. Fortunately, an analyst does not need to painstakingly examine every single byte of a malware sample; only specific portions of a binary typically warrant the deep, instruction-by-instruction scrutiny that reverse engineering demands, and there exists a range of tools designed to help focus that effort where it matters most.

At the most fundamental level, a computer executes a sequence of instructions. Malware, however, often "writes" these instructions in ways that diverge meaningfully from how a standard, benign program would be structured. A malicious sample might employ specific instructions that accomplish a given task in a highly inefficient or unusual way, purely because that unusual approach helps it evade signature-based detection or confuses automated analysis tools.

Disassemblers allow an analyst to examine code starting from the executable's declared entry point — the address, specified within the executable file's header, that dictates which instruction must be executed first when the program is loaded and run. A critical subtlety of the x86 architecture is that instructions are encoded as sequences of bytes, and the interpretation of any given byte depends entirely on the context established by what has already been read and decoded up to that point. Ordinarily, when reading a book, one begins at the first sentence and proceeds linearly to the end. Compiler-generated code behaves in a broadly analogous fashion: control flow, while it does branch and jump, tends to respect clean boundaries between instructions. Malware, by contrast, might begin reading its own instruction stream in an unusual order, jump abruptly to a location that is not the "obvious" next instruction, and then continue reading from what amounts to the middle of what a naive disassembler would have interpreted as a completely different instruction. The effect is somewhat like a reader whose eyes jump mid-sentence to a different point on the page and continue reading from there, deriving a sequence of words that bears no resemblance to what the original, linear reading would have produced. This deliberately anomalous structuring does not arise by accident — it exists specifically because the malware's author does not want the code to be easily analyzed. Malware may go even further and rewrite its own code at runtime, meaning that the bytes present in the file on disk do not even fully represent what will actually execute in memory. Because of these evasive properties, a careful analyst must, at least in principle, be prepared to analyze code one instruction at a time in order to achieve precision, rather than relying on the assumption that instructions will be laid out consecutively and unambiguously.

A useful class of tools that alleviates some of this burden is the family of **decompilers**, such as Ghidra and Hex-Rays, which attempt to reconstruct a high-level, C-like representation of a fragment of binary code. There are certain instruction sequences in assembly that are extremely verbose when read in their raw form but that correspond to easily recognizable, common patterns; decompilers are designed to recognize these patterns and collapse them into more legible high-level constructs. For an analyst working through their first malware sample, however, even with these tools, the process can consume a great deal of time, and it is worth being candid about their limitations: decompilation is a process far from perfect. Information is irretrievably lost during the original compilation step — variable names, precise data types, and other identifiers that a human programmer relied upon simply do not survive translation into machine code, so a decompiler can only ever offer its best guess at reconstructing plausible substitutes. Furthermore, if obfuscation has been deliberately applied to the binary, decompiling the resulting obfuscated code will simply produce an obfuscated, and correspondingly unhelpful, reconstructed source — the decompiler cannot see through obfuscation any more effectively than a human reading raw assembly can. Malware authors are also known to specifically employ anti-analysis techniques engineered to confuse or break decompilers outright. Despite all these caveats, decompilation remains, on balance, very helpful for reverse-engineering professionals, since even an imperfect, obfuscated-looking C-like reconstruction is often faster to reason about than raw assembly.

It is worth stating plainly, as a foundational fact underpinning this entire subject: malicious code is not equivalent to compiler-generated code. Where a compiler produces code following predictable, well-understood idioms optimized for correctness and performance, a human or semi-automated malware-crafting process introduces obfuscation of control and data flows, subtle semantics such as an instruction that deliberately jumps into the middle of another instruction's byte sequence (exploiting exactly the byte-stream ambiguity described above), hand-written instruction sequences such as custom API-hashing algorithms used to resolve system calls without leaving human-readable strings in the binary, self-modifying code, and a range of other decoys designed specifically to mislead automated and manual analysis alike.

CPU and Memory Basics

To reverse engineer code effectively, an analyst must understand the substrate on which that code executes: the central processing unit and its relationship to

memory.

Programmers, when writing high-level source code, think in terms of an essentially unlimited number of variables, each of whatever type is convenient for the problem at hand. The central processing unit, however, offers no such luxury. From the CPU's perspective, it is simply a collection of memory cells of specific, fixed sizes; there exist some cells of larger sizes as well, primarily for optimization purposes in domains like multimedia processing. Within the CPU itself, these cells are known as **registers**, and here the gap between the programmer's abstraction and the hardware's reality becomes especially stark: rather than an unlimited supply of named variables, the programmer working at this level has access to only a small, fixed number of general-purpose registers. On a modern (64-bit) CPU, there are additional special registers that are not directly controlled by the programmer — some exist purely to support debugging, and others are reserved for use by the operating system.

The registers directly usable by ordinary programs are known as the **general-purpose registers** (commonly labeled EAX, EBX, ECX, EDX, and so on in the 32-bit x86 world). The key architectural fact to internalize here is the tension between abstraction and hardware reality: high-level programming languages give the illusion of unlimited storage locations, while the CPU physically provides only a handful of extremely fast storage cells. Because registers offer dramatically faster access than main memory, any piece of data that will be accessed frequently benefits enormously from being kept in a register rather than being repeatedly fetched from and written back to memory.

This tension between what a programmer or compiler wants to express and what the hardware can actually provide is formalized through a contractual abstraction layer called the **Instruction Set Architecture (ISA)**. The ISA is an abstract model that describes precisely what a programmer needs to know in order to program the machine: for x86, this includes the fact that there are eight general-purpose registers, the formats in which arithmetic instructions are expressed, and so on. The ISA functions as an interface, or contract, between software and hardware: humans (or, far more commonly, compilers acting on their behalf) produce ISA-conformant assembly code, and this assembly code is then translated by an assembler into a binary format that the physical hardware can directly consume and execute. Intel originally defined the x86 ISA for its 8086 processor, developed between 1976 and 1978. Over the following decades, later extensions to the architecture have consistently preserved backward compatibility with this original design — a design decision with real consequences for malware analysis,

since even today, most malware samples encountered in the wild are still compiled as 32-bit binaries rather than 64-bit. The x86 ISA is also known by the alternate designation IA-32, and its 64-bit successor is referred to as x86-64, or x64 for short.

The prevalence of 32-bit malware, despite essentially all modern machines running 64-bit CPUs, deserves a moment's reflection. All contemporary machines are built around 64-bit CPUs, but these CPUs remain capable of executing 32-bit software, a capability that exists specifically to preserve backward compatibility with older programs. Windows, in particular, goes to considerable lengths to support the execution of older 32-bit software. This continued support for an older execution mode is convenient for a malware author for a very specific reason: it creates additional gaps and inconsistencies between how 32-bit and 64-bit code is monitored, sandboxed, and defended, gaps that a piece of malware can exploit to evade modern defenses. In addition, there remains a nontrivial fraction of machines in the world — particularly older or specialized systems — that are only capable of running 32-bit software at all, so 32-bit malware retains a broader potential victim pool. For both reasons, it remains convenient, from an attacker's perspective, to write malware that targets the 32-bit execution environment.

Word Size and Endianness

A **word** is understood to be the natural data unit for a given CPU design — the size of data that the processor's arithmetic and logic circuitry is fundamentally built to handle in a single operation. For x86 processors specifically, the native word size is 32 bits. This was not always the case: the original word size for the x86 lineage was 16 bits, and it is precisely for this historical reason, rooted in backward compatibility, that x86 instructions still assume, by default, that a "word" operand is 16 bits long, while a 32-bit operand is instead referred to using the distinct term "dword" (double word). The common IA-32 data type nomenclature, then, comprises byte (8 bits), word (16 bits), and dword (32 bits).

Memory itself, regardless of how large a logical data unit a CPU register might hold, always operates at the byte level — every individually addressable location in memory holds exactly one byte. This raises an immediate practical question: if a register can hold a 32-bit (4-byte) quantity with no difficulty, how does one write that same 32-bit quantity out to memory, which can only address and store data one byte at a time? The answer requires a rule that determines, precisely, in what order the individual bytes composing a multi-byte value should be written to (and

subsequently read from) consecutive memory addresses. This rule is known as **Endianness**.

Consider a concrete example: suppose the goal is to store the dword value 0xABADCAFE at memory address 0x1000. This 32-bit value decomposes into four constituent bytes: AB (the most significant byte, MSB) and FE (the least significant byte, LSB), with AD and CA in between. There are exactly two conventions for how to lay these four bytes out across the four consecutive memory addresses 0x1000 through 0x1003. Under **Big Endian** encoding, the most significant byte is written first, so address 0x1000 would hold AB, followed by AD, CA, and finally FE at address 0x1003. Under **Little Endian** encoding, the convention is reversed: the least significant byte is written first, so address 0x1000 would hold FE, followed by CA, AD, and finally AB at address 0x1003. The x86 architecture uses Little Endian encoding throughout. This detail is far from a mere curiosity for the malware analyst: any time raw memory or a hex dump is being read and interpreted as a multi-byte numeric value or pointer, the bytes must be mentally (or programmatically) reversed to reconstruct the intended value.

Registers in Detail

As already noted, there exists a fundamental mismatch between the programmer's mental model, in which arbitrarily many named variables can be created, and the CPU's physical reality, in which computation is confined to a small number of registers. Registers hold data with extremely fast access time compared to main memory, which is precisely why they are used as the CPU's primary working storage during computation.

Among the registers available on an x86 CPU, several categories are worth distinguishing. The **general-purpose registers** (GPRs) can store either ordinary data values or memory addresses (pointers) — the CPU imposes no rigid distinction between the two; it is purely a matter of how the surrounding code chooses to interpret the bits held within a given register. The **status register** holds a collection of truth values (individual bits, each with a specific meaning) reflecting the state of the processor immediately after the execution of the most recent instruction — this is heavily used, for instance, in evaluating conditional branches. The **program counter** holds the memory address of the instruction currently being executed, effectively tracking the CPU's position within the running program. Beyond these, there exist many other, more specialized registers serving various architectural purposes. On the x86 architecture specifically, the general-purpose registers are 32 bits wide.

Accessing Registers

The x86 architecture provides eight general-purpose registers: commonly referred to by the letters A, B, C, D, along with SI and DI (source index and destination index), plus two registers with special conventional roles. The **ESP** register conventionally points to the top of the stack, and the **EBP** register is often set equal to ESP at the beginning of a function's execution, establishing what is called a base pointer for that function's local data — though, when a function chooses not to use this convention, EBP remains available for general-purpose use like any other register.

The prefix **E** on a register name (as in EAX, EBX, and so on) indicates that the full 32-bit size of the register is being referenced; this E prefix stands for "Extended." When the E prefix is absent, only the least significant 16 bits (that is, the lowest 2 bytes) of the register are manipulated — so, for instance, AX refers to the lower 16 bits of EAX. Furthermore, for the four registers A, B, C, and D specifically, the two least significant bytes of that 16-bit portion can themselves be addressed individually: the high byte of that 16-bit segment is accessed using a name like AH ("A high"), and the low byte is accessed using a name like AL ("A low"). This layered addressing scheme — a full 32-bit register containing a 16-bit sub-register, which in turn contains two individually addressable 8-bit sub-registers — reflects the architecture's historical evolution from 8-bit, to 16-bit, to 32-bit generations, with each new generation extending rather than replacing what came before.

Two particularly important registers cannot be accessed directly by ordinary instructions: the program counter, referred to on x86 as **EIP**, and the status register, referred to as **EFLAGS**. Both are updated implicitly as a side effect of instruction execution rather than being directly writable the way a general-purpose register is.

The Instruction Cycle

The CPU operates according to the classic **fetch-decode-execute cycle**. During the **fetch** stage, the CPU reads the instruction located at the memory address currently held in EIP. During the **decode** stage, the control unit interprets the raw bytes just fetched to determine precisely what operation and operands the instruction specifies. During the **execute** stage, the specified computation is actually carried out, whether that involves the arithmetic logic unit (ALU) performing some calculation or simply moving data between locations. After

execution completes, EIP ordinarily advances to point at the next instruction immediately adjacent in memory — unless the instruction that was just executed explicitly altered the control flow, for instance via a jump, in which case EIP is instead set to whatever destination address the jump specified.

Instructions on x86 vary considerably in their encoded size, ranging anywhere from 1 to 15 bytes, and are stored consecutively in memory along with any immediate operands — meaning literal data values or addresses — that they require as part of their operation. Register operands, by contrast, are not stored as separate bytes following the instruction; instead, they are embedded directly within the binary representation of the instruction's opcode itself. This is illustrated by the following pair of encodings:

```
b8 01 00 00 00     mov eax, 0x1
bf 01 00 00 00     mov edi, 0x1
```

Here, both instructions move the same immediate value (0x1, encoded as the four bytes 01 00 00 00 in little-endian order) into a register, but the specific destination register — EAX in the first case, EDI in the second — is encoded not as a separate operand byte sequence but as part of the opcode byte itself (b8 versus bf). This is a direct illustration of the earlier point that register operands live inside the opcode's own binary representation.

Memory Map of a Process

To fully appreciate how a running program is laid out, it is useful to consider a simplified view of the virtual address space of a 32-bit Windows program. In this simplified model, the lower 2 gigabytes of the address space are reserved for user-space code and data, while the upper 2 gigabytes are reserved for kernel-mode code, inaccessible to ordinary user-mode processes.

Within the user-space portion of the address space, several distinct regions can be identified. At the very base of the address range sits the **stack for the main thread**, a region of memory that, notably, grows toward lower addresses as it is used — meaning that as more data is pushed onto the stack, the boundary of the stack region extends downward, toward address 0x00000000, rather than upward. Adjacent to the stack lies the **heap**, which by contrast grows in the opposite direction, toward higher addresses, as dynamic memory allocations are made. This opposite-direction growth is a deliberate design choice: it allows the two regions to expand toward one another from a shared pool of available address space without

immediately colliding, maximizing the flexibility available to a program that might need substantially more stack space, more heap space, or some mixture of both, depending on its runtime behavior.

Further along in the address space sits the **program image** itself, which comprises the PE (Portable Executable) header along with the code section, imports, and resources bundled into the executable file. Beyond this lies a region that is allocatable as stack space for additional threads or as additional heap, followed by dedicated areas for **DLLs** (Dynamic Link Libraries) that the process has loaded, each of which occupies its own region of the address space. Finally, near the very top of the user-accessible range, fixed addresses are reserved for structures such as the first Thread Entry Block, the Process Entry Block, and a special page that the kernel shares directly with userland code for certain low-overhead communication purposes.

The critical, deliberately emphasized fact from this memory layout is that the **stack grows toward low addresses**. This single detail has significant practical consequences for reading stack-related assembly code and diagrams: when an analyst observes stack addresses in a debugger or in a diagram, addresses that are numerically lower correspond to more recently pushed, "deeper" positions on the stack, while addresses that are numerically higher correspond to older, earlier stack frames closer to the point where the thread began execution.

Memory Protection

The virtual address space described previously is not an undifferentiated expanse in which any code may read, write, or execute at will. The processor and operating system cooperate to enforce rules governing what may be done with each region of memory, and the mechanism that makes this enforcement possible is **paging**.

Under a paging scheme, the operating system divides the entire address space into fixed-size chunks called **pages**, conventionally 4 KB each. Each page carries metadata that answers two distinct questions. First, is this address valid at all — that is, does it correspond to a page that has actually been mapped into the process's address space, or is it unmapped territory that an access would be forbidden from touching? Second, for a page that is valid, what operations are permitted upon it: may it be read, may it be written, may its contents be executed as instructions? These read, write, and execute permissions are enforced at the granularity of regions rather than individual bytes, which is a natural consequence of the page-based organization. A further, security-critical rule layered on top of

this is that pages belonging to kernel memory are accessible only from kernel code; any attempt by ordinary user-mode code to touch kernel pages is rejected outright.

This permission machinery is the foundation upon which a significant class of defensive mechanisms is built. Historically, a program's stack was treated as plain memory on which instructions could, in principle, be placed and executed. This permissiveness became a serious liability, because numerous exploitation techniques depend precisely on the attacker's ability to inject instructions into a writable region such as the stack or the heap and then redirect execution into that injected code. In modern operating systems, therefore, the stack is never executable: the pages comprising it are marked writable but not executable, so that even if an attacker succeeds in writing bytes there, the processor will refuse to interpret those bytes as instructions.

The security measure responsible for this behavior is **DEP (Data Execution Prevention)**. DEP enforces a strict separation between regions intended to hold data and regions intended to hold code: it prevents execution not only on the stack but also on the heap, ensuring that memory meant to store data cannot be hijacked as a staging ground for injected instructions. Operating systems may enforce a suite of such high-level security mechanisms specifically designed to hinder memory-vulnerability exploitation attacks, and DEP is one pillar of that suite.

A second major pillar is **ASLR (Address Space Layout Randomization)**. Rather than loading the various regions of a process — the program image, the DLLs, the stack, the heap — at fixed, predictable addresses on every execution, ASLR deliberately randomizes their placement each time a process is spawned. The practical consequence is directly observable: if the same program is executed twice, the addresses assigned to its regions will differ between the two runs. This randomization extends to dynamically loaded libraries as well — on Windows, where libraries take the form of DLLs, their load addresses are subject to the same per-launch randomization. The heap is likewise affected, so that running a program twice and inspecting the addresses of heap-allocated variables will reveal specific variations between the two executions. The defensive value of this unpredictability is substantial: many exploitation techniques require the attacker to know in advance the precise address of some target (a function to hijack, a buffer to jump into), and if those addresses shift unpredictably on every run, the attacker's hard-coded assumptions break. Beyond DEP and ASLR, more recent Windows versions (Windows 10 and later) introduce additional protections such as

heap allocation randomization and other hardening measures that further complicate the exploitation of memory vulnerabilities.

Memory Addressing

Having established how memory is protected, the next question is how instructions actually specify the memory locations they wish to operate upon. **Addressing modes** provide the vocabulary for expressing the addresses of data to be read from or written to main memory. The expressions used to form these addresses can incorporate immediate operands (literal constant values), registers, or a combination of both, and the real power of addressing modes lies in exactly this combinatorial flexibility: they allow operands to be assembled into complex expressions whose computed value depends on the current program state. Because a register's contents change as the program runs, an address expressed in terms of registers is effectively a dynamic address, resolved freshly each time the instruction executes. One important architectural constraint tempers this flexibility: an instruction can usually reference at most one memory operand, so one cannot, for instance, directly copy from one memory location to another in a single ordinary instruction without routing the value through a register.

A crucial point of interpretation underlies all of this. When an address expression is enclosed in square brackets, it denotes a **dereference** — that is, the instruction operates on the *contents* found at the computed memory location, not on the address value itself. So the instruction

```
mov eax, [0x1000]
```

does not place the number 0x1000 into EAX; rather, it reads the value stored in memory at address 0x1000 and places that value into EAX. Similarly,

```
mov eax, [esi]
```

treats the current contents of the ESI register as a memory address, reads the value stored at that address, and places it into EAX.

Common Addressing Modes

The square-bracket expression can be considerably richer than a bare constant or a single register. The architecture supports a family of addressing modes of

increasing expressive power, several of which are encountered constantly in real code.

The simplest, an **immediate** address, uses a literal constant as the location, as in `mov eax, [0x1000]`. A **register** address uses the contents of a register as the location, as in `mov eax, [esi]`. Adding a constant yields the **register-plus-offset** mode, as in `mov eax, [esp-8]`, which reads from the location eight bytes below wherever ESP currently points. More elaborate still is the **register-scaled-plus-offset** mode, as in `mov eax, [ebx*4+0xff]`, in which the register's value is first multiplied by a scale factor and then offset by a constant. The most general form combines a base register, a scaled index register, and an offset: **base-plus-register-scaled-plus-offset**, as in `mov eax, [edx+ebx*4+8]`. In the last two forms, the offset field is optional.

This most general form is worth dwelling on, because it maps directly onto an extremely common high-level construct: iterating over an array. Imagine a loop walking through an array of 32-bit integers, each of which occupies 4 bytes. In the expression `[edx+ebx*4+8]`, the base register EDX would hold the base address of the array, the index register EBX would hold the current loop index `i`, the scale factor of 4 accounts for the fact that each integer element is 4 bytes wide, and the constant 8 serves as a fixed displacement into the structure. The processor computes the full effective address in a single step — base plus index-times-element-size plus displacement — which is precisely the arithmetic that accessing `array[i]` requires. Recognizing this idiom allows an analyst to reconstruct array accesses from otherwise cryptic bracket expressions.

Basic x86 Instructions

Every program, benign or malicious, is ultimately assembled from three fundamental kinds of instruction: those that **move data**, those that perform **arithmetic and logic**, and those that control **program flow**. Nearly any computation imaginable can be expressed as some arrangement of these three categories.

Data movement instructions are the most straightforward of the three. The workhorse of this category is **mov**, written in the form `mov dest, src`, which copies a value from a source to a destination. The source may be a register, the contents of a memory location, or an immediate constant; the destination may be a register or a memory location. The essential semantics are simple: whatever

value the source denotes is copied into the place the destination denotes, leaving the source unchanged.

The other members of the data-movement family are **push** and **pop**, which exist specifically for manipulating the stack. These two deserve careful attention because they answer a recurring practical problem.

The Problem of Limited Registers and Local Storage

Recall the fundamental tension already established: programmers and compilers reason in terms of abundant named variables, but the CPU provides only a small handful of general-purpose registers. Two situations arise naturally from this scarcity. First, a function may need somewhere to keep its own local working data — storage pertaining to its own execution that must persist across the instructions making up that function. Second, and more acutely, a piece of code may simply run out of free registers: it needs to perform some computation, but every register it could use already holds a value that must not be destroyed.

The solution in both cases is to use the stack as overflow storage. When a register is needed but all of them are occupied, the code can save the current contents of some register onto the stack, freely reuse that register for the computation at hand, and then restore the saved contents afterward — effectively "borrowing" a register for a while. The two operations that make this saving and restoring clean are precisely push and pop.

Concretely, suppose that no free registers remain and the goal is to add to EDX the difference between EBX and ECX, a computation that naturally wants a scratch register to hold the intermediate difference. One approach is to carve out a dedicated temporary memory location, conceptually a slot called TMP, and write:

```
mov TMP, eax
mov eax, ebx
sub eax, ecx
add edx, eax
mov eax, TMP
```

Here EAX is first saved into TMP, then borrowed to compute `ebx - ecx`, the result of which is added into EDX, after which EAX's original value is restored from TMP. The stack offers a more idiomatic way to accomplish exactly the same borrowing, replacing the explicit temporary slot with push and pop:

```
push eax
mov eax, ebx
sub eax, ecx
add edx, eax
pop eax
```

The `push eax` saves EAX's current value onto the top of the stack; EAX is then freely used as scratch space for the subtraction and addition; and `pop eax` restores the original value from the stack back into EAX. The net effect is identical, but the stack-based version requires no pre-arranged temporary location, since the stack itself serves as an ever-available scratch area.

Arithmetic Instructions

Arithmetic instructions carry more analytical weight than their everyday simplicity might suggest, and they are relevant to code analysis for several interlocking reasons. They are used to compute offsets and function addresses, to modify the stack pointer during stack manipulation, and — significantly — they frequently do double duty as data-movement operations or as a means of updating the EFLAGS status register as a side effect of their computation. Because so many instructions update the flags implicitly, an arithmetic instruction is often placed not for the sake of its numeric result at all, but precisely to set the flags in preparation for a later conditional decision.

The core arithmetic instructions and their effects are as follows. The instruction `add dest, src` performs `dest += src`, adding the source into the destination. Its counterpart `sub dest, src` performs `dest -= src`, subtracting the source from the destination. The `inc dest` instruction increments its operand (`++dest`), while `dec dest` decrements it (`--dest`). Two unary instructions produce complements: `not dest` computes the one's complement of its operand (a bitwise inversion of every bit), and `neg dest` computes the two's complement (the arithmetic negation that turns a value into its signed negative).

Logic Instructions

The logic instructions perform bitwise Boolean operations. The instruction `and dest, src` computes a bitwise AND of its operands, `or dest, src` computes a bitwise OR, and `xor dest, src` computes a bitwise exclusive OR. Several common instances of these instructions recur throughout real-world code. A particularly famous idiom is zeroing a register with `xor eax, eax`: since any

value XORed with itself yields zero, this instruction sets EAX to zero, and it is frequently preferred over the more obvious `mov eax, 0` because its encoding is shorter and faster. Logic instructions are also routinely used for extracting specific bits from a value and for checking conditions, and they feature prominently in obfuscation, where XOR in particular is a staple of simple encryption and data-scrambling schemes.

A critical caution must be sounded here, because it is a frequent source of confusion for anyone approaching assembly from a high-level language background. These operations are strictly **bit-wise**: they operate independently on each individual bit position of their operands. This is emphatically *not* the same as the high-level logical operators `&&` and `||` found in languages such as C, which operate on whole truth values and, moreover, typically short-circuit their evaluation. The bitwise `and` and `or` of x86 have entirely different semantics from their high-level namesakes, and conflating the two will lead directly to misinterpretation of code.

Rotate and Shift Instructions

The final family in the arithmetic-and-logic category comprises the rotate and shift instructions, which are frequently encountered in bit-manipulation routines. Shift operations have an important arithmetic interpretation: shifting a binary value left by one position is equivalent to multiplying it by two, and shifting right by one position is equivalent to dividing by two, which makes shifts a fast substitute for multiplication and division by powers of two.

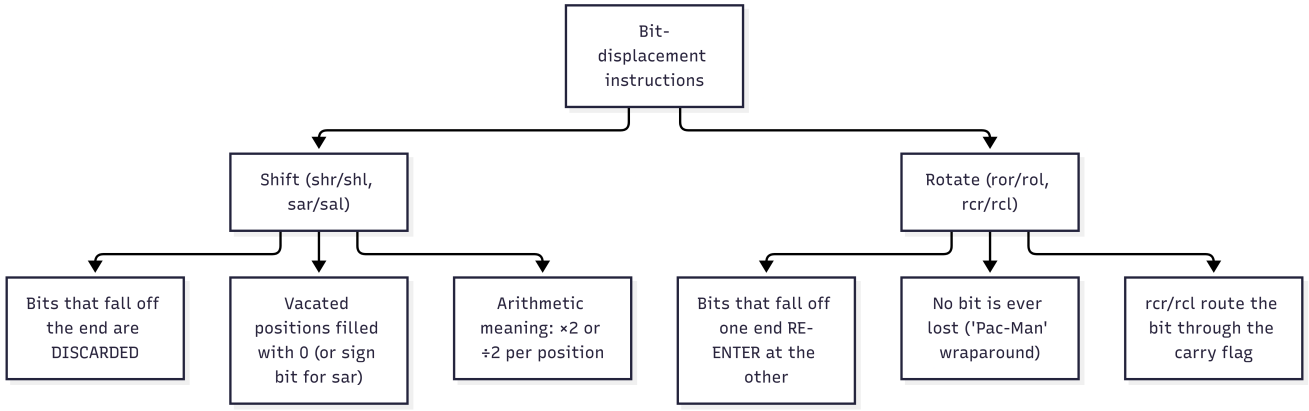
The architecture distinguishes carefully between several variants of these operations, and the distinctions matter.

The unsigned shift instructions are `shr dest, src` and `shl dest, src`, performing an unsigned shift to the right and to the left respectively. In an unsigned shift, bits that fall off one end are discarded, and vacated positions are filled with zeros. The signed shift instructions, `sar dest, src` and `sal dest, src`, perform signed shifts; the signed right shift in particular is designed to preserve the sign of a two's-complement number as it shifts, so that dividing a negative number by a power of two yields the correct negative result rather than a corrupted value.

Rotation differs from shifting in one essential respect: rather than discarding the bits that fall off the end, a rotation feeds them back in at the opposite end. A bit shifted out of the most significant position reappears at the least significant

position, and vice versa — a kind of circular "Pac-Man" wraparound in which no bit is ever lost. The plain rotate instructions are `ror dest, src` and `rol dest, src`, rotating right and left. A further variant, `rcr dest, src` and `rcl dest, src`, performs a rotate *through the carry flag*: the carry bit is treated as an additional bit position participating in the circular rotation, so the bit shifted out goes into the carry flag while the previous carry value is shifted into the operand. For all of these shift and rotate instructions, when the source operand specifying the shift or rotation amount is absent, a default count of 1 is assumed.

The conceptual distinction between discarding bits and recirculating them can be summarized as follows:



Toward Program Flow Control

The third and final category of instruction governs **program flow control** — the means by which a program moves its execution from one point in the code to another rather than simply proceeding to the next adjacent instruction. A foundational fact must be kept firmly in mind when reasoning about any control transfer: by the time an instruction's effect is being evaluated, the program counter has *already advanced* past that instruction to point at the next one. This "pre-advanced" state of the program counter is exactly what makes relative, offset-based jumps work correctly, since offsets are measured from the position the program counter has already reached — the detailed arithmetic of which forms the natural starting point for the study of branching.

Program Flow Control

Control flow is what elevates a program from a mere straight-line list of calculations into something capable of making decisions, repeating work, and organizing itself into reusable units. In x86, control flow can be implemented in three distinct ways, and understanding the differences among them is essential for following the logic of any binary.

The first is the **unconditional branch**: an instruction that replaces the current value of EIP with some desired destination address, causing execution to continue there regardless of any condition. The second is the **conditional branch**, which likewise replaces EIP with a desired address, but only when a condition holds — specifically, a condition evaluated over the value of one or more bits in the EFLAGS register. If the condition does not hold, the branch is not taken, and execution simply falls through to the next adjacent instruction. The third is **function calls and returns**, which are best understood as a special, structured type of unconditional branch, enriched with the extra machinery needed to remember where to come back to once the called routine finishes.

Implementing any of these mechanisms requires two fundamental primitives. The first is a way to specify the target of a branch — the place execution should jump to — which may be given either statically (a fixed address or offset baked into the instruction) or dynamically (an address computed at runtime and held in a register or memory location). The second is a way to evaluate a condition on the program state, so that conditional branches have something to test. As a convenience for human readers and assemblers, code locations in assembly can be annotated with **labels** — symbolic names standing in for addresses — so that a branch can refer to a destination by a meaningful name rather than by a raw numeric address.

Unconditional Branching

An unconditional branch, realized by the `jmp` instruction, can take its destination in one of two broad forms, and the distinction between them is a recurring source of confusion when reading disassembly.

The first form is an **offset**. Here the destination is specified either as a relative offset from the current EIP value, or as an absolute offset measured from the base of the current code segment. The second form is an **absolute address**, supplied directly as a register or memory operand, whose contents are taken to be the exact address to jump to.

The essential subtlety, and the reason offsets cause trouble, stems directly from the fact established earlier: by the time a jump is evaluated, EIP has already advanced past the jump instruction to point at whatever follows it. A relative offset is therefore added not to the address of the jump itself, but to the address of the instruction immediately after it. Consider the following assembly, shown alongside the machine code and the disassembler's interpretation that it produces:

```
                          0:  e9 00 00 00 00    jmp  5
<_main+0x5>
jmp 0x4                   5:  eb 05             jmp  c <L>
jmp L                     7:  83 e8 00          sub  eax,0x0
sub eax, 0                a:  ff e0             jmp  eax
jmp eax          ⟶       0000000c <L>:
L:                        c:  83 c0 00          add  eax,0x0
add eax, 0                f:  ff 20             jmp  DWORD PTR
[eax]
jmp [eax]
```

The instruction most worth tracing is the one at address 5: `eb 05`, written in source as `jmp L`. The opcode `eb` is a relative jump, and its operand `05` is the offset. Crucially, this offset is *not* added to 5 (the address of the jump) but to 7 — the address of the instruction immediately following the two-byte jump, which is where EIP already points once the jump is being carried out. The arithmetic is therefore 7 + 5 = 0xc, which is exactly the address of label L. This is the little piece of arithmetic that trips up newcomers: the result `c` seems not to match a naive "5 plus 5" expectation precisely because the base of the addition is the *already-advanced* program counter, not the jump instruction's own address.

By contrast, the jumps that take absolute addresses are entirely straightforward to read. The instruction at address a, `ff e0` or `jmp eax`, simply transfers control to whatever address is currently held in EAX — no offset arithmetic is involved. Likewise `jmp DWORD PTR [eax]` at address f jumps to the address *stored at* the memory location pointed to by EAX, dereferencing one level of indirection but again requiring no relative computation. The lesson for the analyst is clear: absolute-address jumps are read directly, while offset-based jumps demand the small but critical mental step of adding the offset to the address of the *following* instruction.

Conditional Branching

Unconditional jumps are only half of what control flow requires; the far more interesting half is the ability to jump *only when some condition is met.* Conditional branches accomplish this by evaluating conditions over selected bits of the EFLAGS register. Four flags in particular account for the overwhelming majority of conditional logic encountered in practice.

The **CF (carry flag)** is meant to signal errors — specifically overflows or borrows — arising in *unsigned* arithmetic. The **OF (overflow flag)** plays the analogous role for *signed* arithmetic, signaling when a signed operation has produced a result too large (or too small) to fit in the destination. The **SF (sign flag)** is set whenever the result of an operation is negative, mirroring the value of the result's most significant bit under two's-complement interpretation. The **ZF (zero flag)** is set whenever the result of an operation is exactly zero. These four flags, individually and in combination, encode essentially everything a program needs to know about how its most recent arithmetic turned out.

The flags are rarely useful in isolation; what makes them powerful is the ability to set them deliberately in order to compare two values. This is the purpose of the **cmp** instruction. The instruction `cmp dest, src` computes the difference `dest - src` — but, critically, it does *not* store that difference anywhere, leaving `dest` entirely unmodified. Its sole effect is to set the EFLAGS bits exactly as a real subtraction would have, so that the relationship between `dest` and `src` becomes legible in the flags. For instance, if `dest` and `src` are equal, the subtraction yields zero and ZF is set; if `dest` is less than `src`, the subtraction produces a negative or borrowing result that the sign, carry, and overflow flags collectively capture. Because `cmp` performs this comparison without the side effect of altering its operand, it can be freely placed before a conditional jump to set up a decision without disturbing the values being decided upon. The flags it leaves behind enable a whole family of useful comparisons.

The jcc Family of Conditional Jumps

Reading the flags directly would be tedious, so the architecture provides a large family of **jcc** instructions — "jump on condition code." A jcc instruction inspects the flags for the condition named by its **cc** suffix and jumps to a given offset accordingly. Two properties are worth stressing immediately. First, like the relative jumps discussed above, conditional jumps take an *offset* as their destination; they never take an absolute address. Second, each condition comes paired with a logically negated counterpart, so that the analyst can always express both "jump if the condition holds" and "jump if it does not," choosing whichever reads more naturally.

The meaning of each conditional jump is understood relative to a comparison that has just been performed. Supposing the preceding instruction was `cmp D, S` — comparing a destination value D against a source value S — the principal conditional jumps behave as follows.

The instructions `je L` and its synonym `jz L` jump to label L when `D == S` (equivalently, when the zero flag is set, since equality makes the difference zero). Their negation, `jne L` / `jnz L`, jumps when `D != S`. The pair `jg L` and `jnle L` jump when `D > S` treating the operands as *signed*, with the negated form `jng L` / `jle L` jumping when `D <= S`. The pair `jge L` and `jnl L` jump when `D >= S` (signed), negated by `jnge L` / `jl L`, which jump when `D < S`. For *unsigned* comparisons, a parallel set of mnemonics exists: `ja L` and `jnbe L` jump when `D > S` with the operands interpreted as unsigned ("a" for *above*), negated by `jna L` / `jbe L`; and `jae L` and `jnb L` jump when `D >= S` unsigned, negated by `jnae L` / `jb L` ("b" for *below*).

The signed-versus-unsigned distinction embedded in this mnemonic scheme is not a pedantic detail. The architecture uses an entirely separate vocabulary — *greater/less* for signed comparisons, *above/below* for unsigned ones — precisely because the same bit pattern can represent two different numeric values depending on interpretation, and a comparison that is correct under one interpretation is wrong under the other. The following table consolidates the family:

Jump (and synonym)	**Jump to L if (after `cmp D, S`)**	**Negated version**
`je` L ; `jz` L	D == S	[D != S] `jne` L ; `jnz` L
`jg` L ; `jnle` L	D > S (signed)	[D <= S] `jng` L ; `jle` L
`jge` L ; `jnl` L	D >= S (signed)	[D < S] `jnge` L ; `jl` L
`ja` L ; `jnbe` L	D > S (unsigned)	`jna` L ; `jbe` L
`jae` L ; `jnb` L	D >= S (unsigned)	`jnae` L ; `jb` L

Other Comparisons

While `cmp` is the most direct way to compare two values, it is not the only instruction that sets flags in a way useful for branching, and real-world malware is unlikely to confine itself to the obvious choices.

A second common comparison instruction is **test**. Rather than subtracting, `test` computes the *bitwise AND* of its two operands, and like `cmp` it discards the result, leaving both operands unaltered and affecting only the flags. The most

frequent use of `test` is checking whether a register holds zero. Writing `test D, D` — ANDing a register with itself — produces a result that is zero if and only if D itself is zero, which means the zero flag is set exactly when `D == 0`. A `jz` placed immediately afterward then branches on precisely that condition. This idiom, `test D, D` followed by `jz`, is the canonical way to ask "is this register zero?" and appears constantly.

The analyst must nonetheless remain alert to the fact that malware will not restrict itself to `cmp` and `test` when it wants to make a decision. Because so many ordinary arithmetic instructions update the flags as a side effect, the flags needed for a branch can be produced by an instruction whose apparent purpose is something else entirely. For example, one can decrement a value using `dec` or `sub`, and these instructions will set the zero flag when the result reaches zero; a subsequent `jz` can then branch on that outcome without any explicit comparison instruction ever appearing. This is not merely an obfuscation trick — optimizing compilers do exactly the same thing, folding a comparison into a decrement that was going to happen anyway — so the analyst must read the flag-setting behavior of *every* instruction, not just the ones nominally designated as comparisons.

Intel vs. AT&T Syntax

Before proceeding to the structure of functions, it is important to address a purely notational matter that nonetheless causes real confusion: x86 assembly is written in two major syntax dialects, **Intel** and **AT&T**. The two describe the identical underlying machine instructions but render them very differently on the page. Intel syntax is dominant in the Windows world — the primary environment for the malware under study here — but AT&T syntax is encountered frequently enough (particularly in Unix-derived tooling) that an analyst must be able to read both. The relevant differences fall into four areas.

The first is **parameter order**. Intel syntax writes the destination first and the source second (`mov dest, src`), whereas AT&T reverses this, writing the source first and the destination second. This single difference is responsible for a great deal of misreading, since the very same `mov` line means opposite things depending on which convention is in force.

The second is how **parameter size** is conveyed. In Intel syntax, the size of an operation is derived from the register being used — referencing EAX implies a 32-bit operation, AX a 16-bit one, AL an 8-bit one. AT&T syntax instead attaches an

explicit suffix to the instruction mnemonic: `b` for byte, `w` for word, and `l` for long (dword).

The third concerns **sigils** — the decorations placed on operands. Intel syntax autodetects the nature of each operand from context and uses no special prefixes. AT&T syntax marks operands explicitly: a `$` prefixes immediate constants, and a `%` prefixes register names.

The fourth is the notation for **addresses**. Intel syntax uses the square-bracket expression `[expression]` already discussed, adding a size specifier such as `byte`, `word`, or `dword` when the intended operand size would otherwise be ambiguous. AT&T syntax instead writes memory operands in the form `offset(base, index, width)`, packing the same base/index/scale/displacement components into a parenthesized form.

	Intel	**AT&T**
Parameter order	Destination, then source	Source, then destination
Parameter size	Derived from the register used	Suffix (b, w, l)
Sigils	Autodetect	`$` for immediates, `%` for register names
Addresses	`[expression]`; add byte/word/dword when ambiguous	`offset(base, idx, width)`

Functions and Stack Frames

With the full repertoire of data movement, arithmetic, and control flow in hand, it becomes possible to understand how code is organized into functions — and, just as importantly, how precariously that organization rests on convention rather than hardware enforcement.

A **function**, approximately defined, is a unit of code that manages register values and its own portion of the stack independently of other units. Two practical pressures give rise to this notion of independence. First, when one code unit calls another, the called unit should not carelessly clobber registers that the calling unit was still relying upon; without some discipline here, every function call would risk silently corrupting the caller's working data. Second, since CPUs come equipped with only a few general-purpose registers, a function frequently needs more

working storage than registers can supply, and the standard remedy is to **spill** values — that is, to save them onto the stack and fetch them back later in the execution as the need arises. Both pressures point toward giving each active function a private, well-delimited region of the stack to work in.

That region is the **stack frame**. Conceptually, the stack is divided into frames, one per currently active function, so that at any moment the chain of frames mirrors the chain of calls that led to the present point of execution. A typical frame bundles together three categories of information. First, it holds the **arguments** that were passed to the function for this particular invocation. Second, it holds the function's **local variables** and any other temporary storage the function requires during its execution. Third, and indispensably, it holds the **return address** for the call — the record of where, in the calling function, execution must resume once this function finishes and exits. It is this stored return address that allows the structured "call and return" discipline to work at all, and, as will become evident, it is also precisely the piece of saved state that memory-corruption attacks most often seek to overwrite.

Local Storage

The notion of a function spilling register values onto the stack, introduced in the abstract, can now be made concrete. Recall the fundamental problem: suppose no free registers remain, yet a computation must be performed that requires a scratch register. The specific example already encountered — adding to EDX the difference between EBX and ECX, which needs an intermediate register to hold the difference — serves equally well here, and it illuminates exactly how the stack substitutes for a scarce register.

The naive approach dedicates a named temporary memory location, call it TMP, which serves as a holding area for whatever value currently occupies EAX, so that EAX may be "borrowed" for a while and then returned to its prior state:

```
mov TMP, eax
mov eax, ebx
sub eax, ecx
add edx, eax
mov eax, TMP
```

The idiomatic stack-based form replaces the hand-managed TMP slot with a push at the start and a pop at the end:

```
push eax
mov eax, ebx
sub eax, ecx
add edx, eax
pop eax
```

Here TMP is, in effect, a temporary location for saving and restoring the data currently in EAX, so that the code can borrow EAX for the span of the three middle instructions and then hand it back untouched. What makes the stack version superior is that it demands no pre-arranged, named storage: the stack is always present and always ready to accept another saved value.

It is worth watching the stack pointer behave as this executes, since the mechanics reveal exactly what push and pop do to ESP. Before the `push eax`, ESP points at the current top of the stack — in the illustrative layout, at the slot holding the value 0x1032, with the three slots above it empty. Executing `push eax` causes ESP to move upward in the diagram (toward lower addresses, as the stack always grows) and the saved EAX value to be written into the newly occupied slot directly above 0x1032. The middle instructions then run, freely trashing and reusing EAX. Finally `pop eax` reads that saved value back out of the slot ESP points to, restoring EAX, and moves ESP back down to where it began, leaving the stack exactly as it was found. The symmetry is total: whatever push disturbs, the matching pop restores.

Push and Pop Instructions

Having seen push and pop in service of register borrowing, it is worth stating their precise semantics, because the exact sequence of micro-steps each performs — and the fact that each can be decomposed into simpler instructions — is something an analyst must be able to recognize on sight.

The instruction **`push src`** performs two actions in order. First, it moves the top of the stack *up* by 4 bytes — meaning it subtracts 4 from ESP, since the stack grows toward lower addresses. Second, it copies the contents of its operand, which may be an immediate or a register, into the memory location now pointed to by the freshly updated ESP. The operand is thereby deposited at the new top of the stack.

The instruction **`pop dest`** performs the mirror image. First, it copies the 4 bytes currently pointed to by ESP into the destination register. Second, it moves the top

of the stack *down* by 4 bytes — adding 4 to ESP — thereby discarding the slot that was just read. The order matters: the value is read out *before* ESP is adjusted, so that pop retrieves the value that push most recently deposited.

A revealing fact about these two instructions is that neither is truly primitive. Each can be realized by a short sequence of simpler instructions combining explicit stack-pointer arithmetic with an ordinary memory move. A `push ebp` is equivalent to subtracting 4 from ESP and then writing EBP to the top of the stack; a `pop edx` is equivalent to reading the top of the stack into EDX and then adding 4 to ESP. The following equivalence makes this explicit:

```
push ebp              sub esp, 4
xor ecx, ecx    ⟶     mov [esp], ebp
pop edx               xor ecx, ecx
                      mov edx, [esp]
                      add esp, 4
```

The `xor ecx, ecx` in the middle is incidental — it merely demonstrates that unrelated instructions may sit between the push and the pop — but the surrounding translation is the point. This decomposition matters enormously to the analyst for one reason above all: malware authors frequently use exactly this substitution to *hide* stack manipulation. A `push` or `pop` is immediately recognizable and draws attention; an equivalent `sub esp, 4` followed by a `mov [esp], ...` accomplishes the same thing while looking like ordinary, unremarkable arithmetic and data movement. An analyst who does not recognize the expanded form as a disguised stack operation will misread the code's intent.

Using the Base Pointer

The stack pointer ESP is a restless register: it moves every time anything is pushed or popped, which happens constantly throughout a function's execution. This restlessness creates a practical difficulty. When a function makes use of local variables stored in its stack frame, those variables need to be addressed somehow — and addressing them relative to ESP means chasing a moving target, since the very same local variable sits at a different ESP-relative offset at different points in the function, depending on how much has been pushed since the frame was established.

The remedy is to dedicate a second, stable register to the job: the base pointer, **EBP**. Upon entering a function that uses local variables, EBP is set to a fixed

anchor point within the frame, after which every local variable can be referenced via a fixed offset from EBP that never changes for the duration of the function, no matter how ESP wanders. Consider the following prologue and subsequent access:

```
mov ebp, esp
sub esp, 8
mov [ebp-8], 0x1111
mov [ebp-4], 0xabadcafe
...
mov eax, [ebp-8]
```

The sequence begins by capturing the current stack top into EBP with `mov ebp, esp`, fixing the anchor. The `sub esp, 8` then lowers ESP by 8 bytes, carving out 8 bytes of room on the stack for two 4-byte local variables. Those two locals are now addressable at the fixed offsets `[ebp-8]` and `[ebp-4]`: the first is initialized to 0x1111 and the second to 0xabadcafe. Crucially, much later in the function — after arbitrary further stack activity has moved ESP around — the first local can still be retrieved with the identical expression `mov eax, [ebp-8]`, because EBP has not budged. Tracing the stack state confirms the arrangement: before the prologue ESP points near 0x3424; after `mov ebp, esp` and the `sub esp, 8`, EBP anchors at the original top (now holding, in the lower slots, 0xabadcafe and above it 0x1111) while ESP has descended two slots below it.

This illustrates precisely why EBP exists: accessing local variables through ESP is like chasing a moving target, whereas EBP provides a fixed frame of reference. There is, however, an important caveat for the analyst. EBP is an ordinary general-purpose register, and nothing compels a function to reserve it for this base-pointer role. If a function's code logic chooses to use EBP for some other purpose — as malware and aggressively optimized code both often do — then the convenient fixed-offset addressing is unavailable, and dereferencing local variables via `ESP+offset`, moving target and all, becomes the only option left. Recognizing whether EBP is or is not being used as a frame pointer is thus one of the first orienting judgments an analyst makes when reading a function.

Call and Ret Instructions

Function calls were earlier described as a special kind of unconditional branch, and the two instructions that implement them — **call** and **ret** — are where that "special" machinery lives. Their entire reason for existing is to solve the problem a

plain jump cannot: a plain jump transfers control but leaves no record of where control came from, so there is no way to return.

A piece of code invokes a function using the **`call`** instruction. Unlike a jump, `call` does something extra *before* the control transfer takes place: it stores on the stack the address of the instruction that immediately follows the `call` itself. This stored address is the **return address** for the call — the place where execution should resume once the called routine is done. With the return address safely on the stack, `call` then transfers control to the target function.

The called function (the callee) resumes the caller by executing **`ret`**. This instruction reads the address currently sitting at the top of the stack and populates EIP with it, causing execution to continue at exactly the instruction after the original `call`. In essence, `ret` is a jump whose destination is whatever address `call` left on the stack. The two instructions are thus a matched pair: `call` pushes the return address and jumps in; `ret` pops the return address and jumps back.

A conceptual warning accompanies all of this, and it is one the analyst must internalize deeply: **functions exist only at a logical level.** The processor has no innate concept of a "function" as a bounded entity. Code is simply laid out in memory as a continuous stream of bytes with no separators, no markers, and no inherent boundaries delimiting where one function ends and the next begins. The division of code into functions is a human and compiler convention imposed *on top of* an undifferentiated byte stream. The consequence for malware analysis is severe: adversaries can deliberately scramble the layout of their code — interleaving pieces of different logical functions, placing a function's body far from its entry, jumping in and out of regions in unexpected ways — making it genuinely difficult for an analyst (and for automated tools) to identify where function boundaries actually lie.

A Worked Function Call

Following the stack through a concrete call makes the matched behavior of `call` and `ret` unmistakable. Consider a caller that executes some instructions and then calls a function named `myfun`:

```
push eax            myfun:
add eax, 1          mov ebp, esp
call myfun          add esp, 8
```

```
sub eax, 1          ...
                    sub esp, 8
                    ret
```

Before the `call myfun` executes, ESP points at some stack top (in the illustration, the slot holding 0x2300). When `call myfun` runs, it pushes the return address — the address of the `sub eax, 1` instruction that follows the call — onto the stack, so ESP now points one slot higher (toward lower addresses) at a slot holding `<return address>`, and control transfers into `myfun`. The callee runs its body, and when it reaches `ret`, the instruction reads that `<return address>` from the top of the stack, loads it into EIP, and execution jumps back to `sub eax, 1` in the caller. The stack is left as it was before the call, ESP once again pointing at the 0x2300 slot. The arrows of the original diagram capture this round trip precisely: control dives from the `call` into `myfun`, and `ret` sends it back up to the instruction immediately after the call, with the stored return address serving as the thread connecting the two.

Calling Conventions

The `call` / `ret` pair explains *how* control gets into and out of a function, but it leaves two equally important questions unanswered. First, how are register values preserved across a function invocation, so that the caller's data survives the call? Second, how are parameters actually passed from caller to callee? Neither question has a single hardware-mandated answer; instead, the answers are fixed by agreement, and those agreements are called **calling conventions**.

A calling convention regulates several distinct matters at once. It specifies how parameters are passed — whether via registers, via the stack, or some combination — and in which order they are placed. It specifies which registers the callee is obligated to preserve on behalf of the caller, so that both sides know whose responsibility it is to protect any given register's value. And it specifies who performs the realignment of ESP upon return — that is, who is responsible for cleaning up the stack space that the arguments occupied once the call completes.

For the malware analyst, a nuanced point follows. Malware is free to use entirely custom, idiosyncratic calling conventions for its own internal functions, since those functions only ever call one another and can agree among themselves on any scheme they like. However, the moment malware needs to interact with the Windows API — which it almost always does, for anything from file access to network communication — it is forced to follow the *standard* conventions that

those system functions expect. This creates a useful analytical foothold: calls into Windows APIs will obey recognizable conventions even when the surrounding custom code does not.

Windows Conventions

Within a single Windows program, different calling conventions can coexist, with different functions obeying different rules. Two conventions dominate.

The first is **cdecl**, the convention used by C programs. Under cdecl, arguments are pushed onto the stack in **right-to-left order**, meaning the last argument in the source-level call is pushed first and the first argument ends up on top, nearest the callee. Register preservation is split as follows: the caller is responsible for saving EAX, ECX, and EDX if it is using them (these three are therefore free for the callee to clobber), while the callee must preserve EBP, EBX, EDI, and ESI whenever it needs to use them. Under cdecl, the **caller adjusts ESP after the return**, cleaning up the pushed arguments itself. A 32-bit return value is delivered back to the caller in the EAX register.

The second is **stdcall**, the convention used by the Win32 API. In stdcall, the treatment of arguments, the division of caller-save and callee-save registers, and the return-value location are all identical to cdecl. The one decisive difference concerns stack cleanup: under stdcall, the **callee** adjusts the stack pointer upon return, rather than the caller. This is accomplished with the `ret N` instruction, where N indicates that N+4 bytes should be added to ESP *after* EIP has been updated with the return address — the extra 4 accounting for the return address slot itself, and the N accounting for the argument bytes the callee is cleaning up on the caller's behalf.

Example: cdecl

A concrete cdecl call makes the division of labor visible. Consider calling a function `subtract` corresponding to the C source `int __cdecl subtract(int arg1, int arg2) { return arg1 - arg2; }`:

```
mov ecx, 20             subtract:
...                     push ebp
push ecx                mov ebp, esp
push 10                 mov eax, [ebp+8]
push 30                 mov edx, [ebp+12]
```

```
call subtract            sub eax, edx
add esp, 8               pop ebp
pop ecx                  ret
cmp eax, ecx
jnz ErrorLabel
```

Reading the caller: ECX (holding 20) is first saved with `push ecx`, since ECX is a caller-save register the caller wants to preserve across the call. Then the two arguments are pushed right-to-left — `push 10` (arg2) followed by `push 30` (arg1) — so that arg1=30 ends up nearer the top. After `call subtract` pushes the return address and transfers control, the callee establishes its frame (`push ebp`, `mov ebp, esp`), reads its two arguments from fixed EBP offsets — arg1 at `[ebp+8]` and arg2 at `[ebp+12]`, the offsets reflecting that `[ebp]` holds the saved EBP and `[ebp+4]` holds the return address — computes `eax = 30 - 10`, tears down its frame with `pop ebp`, and returns the result in EAX via a plain `ret`. Back in the caller, the defining cdecl behavior appears: `add esp, 8` is the *caller* cleaning up the 8 bytes of arguments it had pushed. The subsequent `pop ecx` restores the saved ECX, and `cmp eax, ecx` / `jnz ErrorLabel` checks the returned result.

Example: stdcall

The stdcall version of the identical function, `int __stdcall subtract(int arg1, int arg2)`, differs in exactly one place:

```
mov ecx, 20              subtract:
...                      push ebp
push ecx                 mov ebp, esp
push 10                  mov eax, [ebp+8]
push 30                  mov edx, [ebp+12]
call subtract            sub eax, edx
add esp, 8   ← removed   pop ebp
pop ecx                  ret 8
cmp eax, ecx
jnz ErrorLabel
```

Everything on the caller side up to and including the call is identical. The difference is twofold and complementary. In the callee, the plain `ret` has become **`ret 8`**: after loading EIP with the return address, the callee adds 8+4 bytes to

ESP, itself disposing of both the return address slot and the 8 bytes of arguments. Correspondingly, in the caller, the `add esp, 8` that cdecl required is now *gone* (struck through) — it would be redundant and indeed harmful, since the callee has already performed that cleanup. The stack diagrams for the two versions are identical right up until the return; only at the `ret` does stdcall's `ret 8` perform the extra ESP adjustment that cdecl deferred to the caller. Spotting whether cleanup happens at the call site (`add esp, N` after the call) or inside the callee (`ret N`) is therefore a reliable way for an analyst to tell which of the two conventions a given function follows.

Caller-Save vs. Callee-Save

Underlying both conventions is a general principle for dividing the responsibility of preserving registers, and it is worth stating in its own right because it recurs throughout all of reverse engineering.

Caller-save registers are those that the *caller* must push to the stack *before* making a call, if and only if the caller intends to use their values again after the call returns. The logic is that these registers are considered fair game for the callee to overwrite, so any caller who cares about their contents must protect them itself beforehand. Under both cdecl and stdcall, the caller-save registers are typically EAX, ECX, and EDX.

Callee-save registers are the complement: these are pushed to the stack in the *prologue* of the *callee*, but only when the callee actually needs to use them. The callee, having borrowed them, is obligated to restore them before returning, so that from the caller's perspective these registers emerge from the call unchanged. Under both cdecl and stdcall, the callee-save registers are typically EDI, ESI, and EBX.

A performance observation explains why code tends to prefer one over the other. Compilers often favor callee-save registers for holding values that must survive across nested calls, because a value parked in a callee-save register is automatically protected by whatever functions get called in the meantime. Only when register pressure grows high — when there simply are not enough registers to go around — do compilers additionally press caller-save registers into service. In custom, hand-written malware code, however, no such tidy preference can be assumed: one may find any sort of assortment, since the author is bound by no compiler's habits.

EBP — Enter and Leave

The prologue and epilogue sequences that set up and tear down a stack frame are so stereotyped that the architecture provides single instructions to encapsulate each.

When EBP is used as a base pointer, a function prologue commonly performs:

```
push ebp
mov ebp, esp
```

This pair — saving the caller's EBP, then anchoring EBP to the current stack top — can be accomplished with the single instruction **`enter`**. The `enter` instruction additionally accepts an optional immediate operand specifying an amount to subtract from ESP, carving out room for local variables in the same stroke (equivalent to a following `sub esp, N`).

The symmetric teardown is governed by a strict requirement: to leave a function cleanly, **ESP must point to the caller's return address** at the moment `ret` executes, since `ret` takes whatever ESP points at as the address to return to. If ESP has been left pointing elsewhere (for instance, still down among local variables), EBP can come to the rescue in the function epilogue:

```
mov esp, ebp
pop ebp
```

The `mov esp, ebp` snaps ESP back up to the frame's anchor (discarding all locals at once), and `pop ebp` then restores the caller's saved EBP while advancing ESP to point exactly at the return address, ready for `ret`. This two-instruction epilogue can likewise be collapsed into the single instruction **`leave`**. Recognizing `enter` and `leave` — and their multi-instruction equivalents — lets an analyst immediately identify frame setup and teardown even when a compiler or author has chosen one encoding over the other.

LEA Instruction

The **LEA (Load Effective Address)** instruction occupies a peculiar and frequently misread niche. LEA evaluates an expression that defines a memory address and writes *that address* into a destination register. The crucial contrast is with ordinary data-movement and ALU instructions: those dereference a given address and operate on the *contents* found at the pointed-to location, whereas

LEA does not dereference at all — it simply *computes an address* and hands back the number. Despite borrowing the square-bracket syntax associated with memory access, LEA never actually touches memory; it performs only the address arithmetic.

This gives LEA two distinct uses. The first is genuine **address calculation**, the pointer arithmetic needed to figure out where some data lives without yet reading it. The second, and often surprising, use is as a compact way to evaluate **arithmetic expressions** that have nothing to do with memory at all, exploiting the fact that the address-arithmetic engine can compute `base + index*scale + offset` in a single instruction.

LEA: An Example

The distinction between LEA and MOV is sharpest in a side-by-side comparison. Suppose EBX = 0x1000, and the value 0xABADCAFE happens to be stored at address 0x1004. Then `mov eax, [ebx+4]` *dereferences* the computed address 0x1004 and copies its contents — 0xABADCAFE — into EAX. By contrast, `lea eax, [ebx+4]` computes the same address 0x1004 but copies *the address itself*, 0x1004, into EAX, never reading what lies there. Same bracket expression, entirely different result: one yields the stored value, the other yields the location.

The arithmetic-expression use is illustrated by a computation with no pointers in sight. Suppose EAX = 10 and EBX = 4, and the goal is to compute C = A + 2·B − 1. The single instruction `lea ecx, [eax+2*ebx-1]` produces exactly this result — 10 + 2·4 − 1 = 17 — in one stroke, treating the registers as plain numbers and the "address" expression as ordinary algebra. One constraint governs this trick: the scale factor used for the multiplication must be 1, 2, or 4 (the hardware-supported scaling widths), so LEA can cheaply multiply by these particular small constants but not by arbitrary ones.

REP Instructions

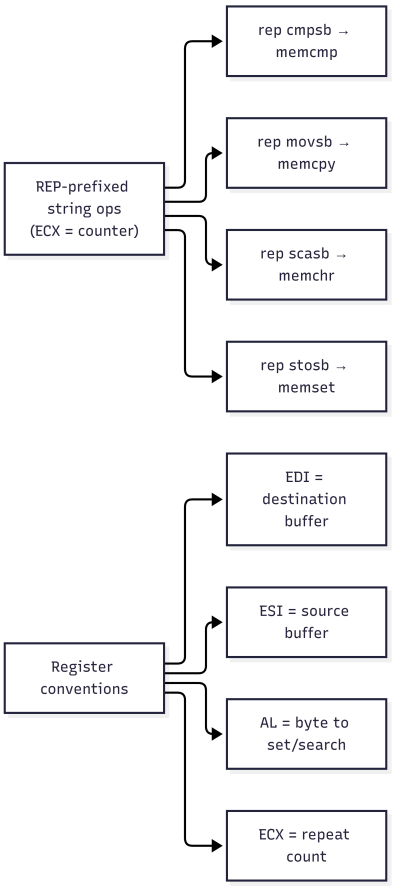
A final family of instructions provides compact, hardware-accelerated primitives for manipulating whole buffers of bytes, and recognizing them lets an analyst instantly identify common memory operations that would otherwise sprawl across an explicit loop.

The base instructions operate on individual bytes and come in four flavors. **`cmpsb`** compares bytes from two buffers. **`stosb`** initializes bytes in a buffer, filling them all with the same value. **`movsb`** copies one buffer into another. **`scasb`** searches a

buffer for a particular byte. Each of these processes a single byte per execution; what turns them into bulk operations is the `REP` prefix placed in front of them. REP causes the instruction to repeat — advancing through the buffer — until either the ECX register, used as a counter, reaches zero, or (for the comparison and search variants) an additional condition on the zero flag is satisfied.

These string instructions draw their operands from a fixed set of registers by convention rather than from explicit operands. EDI holds the address of the first buffer, treated as the **destination**; ESI holds the address of the second buffer, treated as the **source**; AL holds the single byte to be set or searched for; and ECX serves as the repetition counter. Committing this register assignment to memory (destination in EDI, source in ESI, byte in AL, count in ECX) is essential, because the instructions mention none of it on the page.

The power of these primitives is that the entire family of core C standard-library memory functions can be implemented using nothing more than these four REP-prefixed instructions. Specifically, `memcmp` (compare two regions) corresponds to `rep cmpsb`; `memcpy` (copy a region) corresponds to `rep movsb`; `memchr` (search a region for a byte) corresponds to `rep scasb`; and `memset` (fill a region with a byte) corresponds to `rep stosb`. When an analyst encounters one of these REP idioms, the correct mental translation is immediately to the corresponding named library function, collapsing what looks like obscure byte-shuffling into a single recognizable operation.



CHAPTER 2: Static Malware Analysis

Why Analyze Malware?

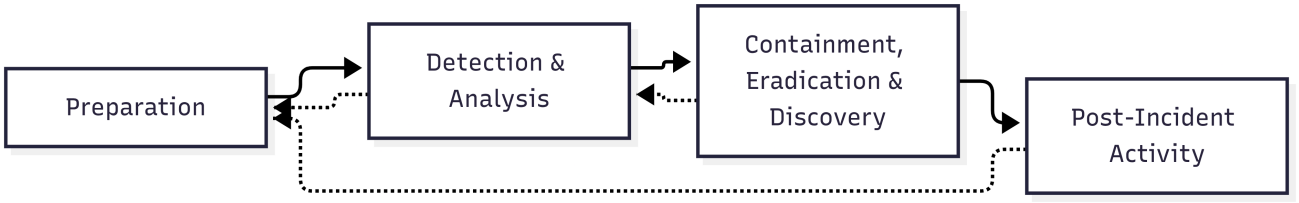
Malware analysis is the disciplined process of dissecting malicious software to understand its inner workings, its intentions, and its footprint on a compromised system. A piece of malware, by design, is a silent executable: once launched it produces no self-description of what it is doing, because it is engineered to leave as few traces as possible and to remain undetected for as long as it can. The analyst's job is to force that silence to give up its secrets. There are three principal motivations that justify the effort.

The first motivation is **monitoring and attack prevention**. By studying how malware operates, an organization can identify recurring attack patterns and translate them into detection rules that are then embedded into automated defensive devices such as intrusion detection systems. Understanding the mechanisms an attacker relies upon — for instance the exploitation of insufficient isolation between components — allows defenders to close exactly those gaps and to design IT systems that are structurally more resilient to forthcoming attacks. In short, comprehending an adversary's strategy is the precondition for building better defenses against it.

The second motivation is **incident response**. When an infection has already taken place, the defender needs to reconstruct precisely what happened. Because the malware advertises nothing about its activity, analysis is the only way to determine which assets — machines, files, accounts — have actually been compromised. This knowledge is essential to contain the further expansion of the attack, to eradicate it completely, and finally to recover efficiently. Without an accurate picture of the malware's behavior, containment and eradication become guesswork, and residual infected components may allow the attack to resurface.

The third motivation is **attribution**: understanding who the attacker is. The internal characteristics of a sample — coding style, reused components, referenced infrastructure — can help tie an incident to a known threat actor.

These three purposes are not isolated academic exercises; they map directly onto the recognized lifecycle of handling a security incident. The **NIST Incident Response Cycle** organizes this lifecycle into four connected phases that flow into one another and loop back for continuous improvement:



Preparation establishes the tools, procedures, and safe environments needed before an incident occurs. Detection and analysis is where malware analysis proper lives: the incident is recognized and the malicious artifact is understood. Containment, eradication, and discovery act on that understanding to stop the spread, remove the threat, and uncover any additional compromise. Post-incident activity closes the loop by feeding lessons learned back into preparation and detection, strengthening the organization against the next attack. Malware analysis is therefore a critical part of incident response, not a peripheral one.

What We Want to Extract: Signatures

The output of an analysis is not necessarily a sixty-page narrative report. Very often the practical deliverable is far more compact: a **signature**, understood as a set of information whose presence within a network or on a host reliably indicates the presence of the malware. Signatures — also called **Indicators of Compromise (IoCs)** — are what allow us to detect the same threat again

elsewhere and to prevent further similar infections. Signatures come in three distinct shapes, each observing the malware from a different vantage point.

Host-based signatures describe artifacts that can be checked on a single machine. They identify files, processes, or registry keys on a victim computer whose presence betrays an infection. These are the traces the malware leaves behind on the endpoint itself.

Network signatures detect the malware by analyzing network traffic rather than local artifacts. They answer questions such as whether the malware inspects its surroundings and, crucially, whether it contacts an external **Command & Control** server — abbreviated **C&C** or **C2** — the remote infrastructure from which many malicious programs receive orders. Network signatures become considerably more effective when they are coupled with the insight gained from malware analysis, because analysis reveals what the traffic should look like and where it is destined.

Behavioral signatures shift the focus from the malware object to its effects: they describe what the malware does to the system rather than what the malware is. This is the domain of behavioral analysis, and it is conceptually different from the classic antivirus signature, which tries to match the static content of the file. A behavioral signature can catch a threat even when its binary form has changed, precisely because the behavior remains constant while the bytes may not.

A Necessary Warning: Handling Malware Safely

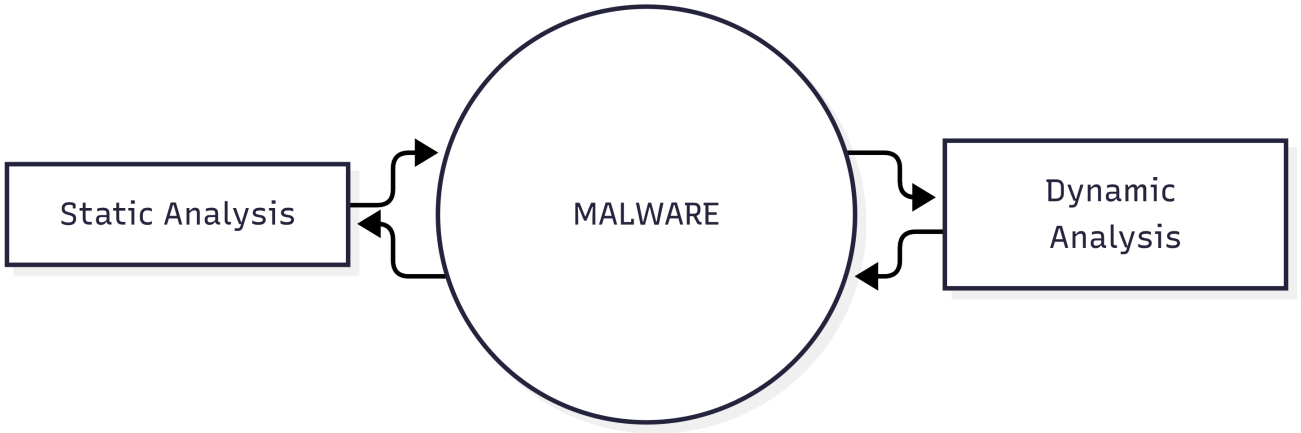
Before any technique is applied, the analyst must internalize that this work is inherently dangerous. The subject of study is software deliberately built to harm computers — possibly the very computer performing the analysis. In many cases the sample must actually be executed to be understood, meaning the analyst voluntarily runs code written by criminals. Furthermore, some IP addresses referenced inside a sample may be live, and one must assume that behind such an address sits a real adversary.

From these premises follows a strict safety discipline. Analysis must always be performed inside a **virtual machine isolated from the main host**, so that any damage is confined to a disposable environment. That virtual machine should be **disconnected from the network** unless network communication is genuinely required by the specific experiment. If references to external IP addresses are found in the code, one must **never** attempt to contact them directly — doing so is comparable to a burglary victim traveling to the burglar's home address to ask

how the break-in was done, an act that only exposes the investigator. And overriding everything, repeated deliberately because it is the single most important precaution: **keep a safe offline backup of your important data**, so that even a catastrophic escape of the malware cannot cause irrecoverable loss.

Static versus Dynamic Analysis

The techniques for extracting information from a malicious executable divide along a fundamental axis: whether or not the sample is run. A malware sample is an executable file containing binary code, written and compiled for a specific processor architecture, and it can be approached in two complementary ways that together form a cyclical, mutually reinforcing process.



Static analysis examines the malware *without running it*, inspecting the code and the structure of the file directly. Its instruments include the `strings` utility, PE inspection tools, and, at the more advanced level, a disassembler such as **IDA Pro** used to interpret assembly code.

Dynamic analysis does the opposite: it *runs* the malware and monitors the effects it produces on the running system. Because execution is involved, it relies on a virtual machine and the practice of taking snapshots so that a clean state can always be restored. Its instruments include monitoring tools such as **RegShot**, **Process Monitor**, **Process Hacker**, and **CaptureBAT**, as well as debuggers; for the examination of volatile memory it uses RAM-analysis frameworks such as **Mandiant Redline** and **Volatility**. Other forms of analysis exist beyond these two families, but they fall outside the scope of this treatment.

The two approaches feed each other: findings from static inspection guide what to watch for during execution, and observations from execution direct where to look in the code. Neither is complete on its own.

Basic versus Advanced Techniques

Cutting across the static/dynamic distinction is a second axis of difficulty, separating basic from advanced techniques.

Basic static analysis observes the malware without looking at its code in depth — relying instead on strings, PE inspection, and similar surface-level tools. It is quick and easy, but it fails against advanced malware and can miss important behaviors, since a determined author can hide precisely the details that matter. **Basic dynamic analysis** is likewise easy but demands a safe test environment, and it too is not effective on every malware, because some samples detect the observation and refuse to reveal themselves.

The advanced counterparts trade speed for depth. **Advanced static analysis** consists of reverse-engineering the malware code with a disassembler; it is complex and requires a solid understanding of assembly language and of how such code interacts with the operating system. **Advanced dynamic analysis** runs the code inside a debugger, allowing the analyst to examine the internal state of the running malicious executable and to watch how that state evolves instruction by instruction. This present material concentrates on the basic techniques, which are the natural entry point.

Expected Behaviors of Malware

Regardless of its specific purpose, malware tends to exhibit a recognizable repertoire of behaviors. Anticipating them lets the analyst know what to look for.

Persistence is the malware's drive to survive. After installing itself in the system, it performs actions that guarantee its survivability across events that would otherwise remove it — most notably reboots, but also inspections and other disruptions. To achieve this it often modifies the operating system configuration, and it takes care that these changes remain hidden from ordinary observation.

Uniqueness reflects that malware is often incompatible with itself. Multiple concurrent executions of the same sample are usually meaningless and can even be dangerous to the malware's own goals — imagine two ransomware instances competing to encrypt the same resources and corrupting each other's work. To avoid this, malware commonly performs checks at startup that guarantee only a single copy is running at any time. It is worth stressing that malware code is not written to be elegant; correctness of self-management, not beauty, is the concern.

Environment checks / Targeting capture the fact that modern malware is not meant to run on every machine it reaches. It is designed to operate only in an

appropriate environment, and it verifies at startup that specific execution conditions are satisfied before acting. These conditions may include the current time or date, the system's localization, the availability of particular resources, or the presence of certain other processes. If the conditions are not met, the malware may stay dormant or exit.

Obfuscation and evasion describe the malware's effort to remain undetected for as long as possible by hiding both its presence and its effects. It may inject its code into other processes to conceal its true identity, it may mangle its own internal code to erase easily discoverable hints of its malicious nature, and it may deploy techniques specifically intended to hamper any attempt to analyze its behavior correctly.

Fingerprinting & beaconing refer to reconnaissance and announcement. The malware inspects the infected system and collects information about it — a fingerprint — and this information can then be used to "publicize" the malware's presence to a remote server, effectively announcing a successful infection.

Communication and control closes the loop with the attacker's infrastructure. Malware frequently interacts with external command-and-control services in order to receive instructions and commands, to update its own internal components, and to exfiltrate stolen data. This communication may travel over heterogeneous types of links, and the data flowing on these channels is often obfuscated to resist inspection.

General Rules for Malware Analysis

Two pragmatic principles should govern the analyst's mindset. The first is **don't get caught in details**: it is neither necessary nor efficient to understand one hundred percent of the code, and the effort should concentrate on the key features that reveal what the sample does. The second is **try several tools**: if one tool fails to yield results, another should be tried, and one should not become stuck on a single hard issue but rather move along and return later if needed. Underlying both rules is the sober reality that malware authors are constantly raising the bar, so the analyst must remain flexible and outcome-oriented rather than perfectionist.

The Techniques of Basic Static Analysis

Basic static analysis, the focus of what follows, rests on a small set of concrete techniques: antivirus scanning, the computation of cryptographic hashes, and the

examination of a file's strings, functions, and headers. Each of these is developed in turn.

Antivirus Scanning: An Obvious First Step

Submitting a suspicious file to antivirus engines is the natural opening move, and services such as **VirusTotal** make this convenient by accepting an uploaded file or URL, analyzing it against many engines, and returning a detection score together with community sharing of the sample. Yet this convenience carries two caveats. First, malware can easily change its signature and thereby fool signature-based antivirus detection, so a clean score is far from a guarantee of safety. Second, uploading a sample to a public service such as VirusTotal may itself alert the attackers that their malware has been caught, since they can monitor these platforms for the appearance of their own creations. The first step is obvious, but it is not free of consequences.

Hashing Samples

A **hash** condenses a file of any size down to a fixed-length fingerprint, computed with algorithms such as **MD5** or **SHA-1**. Because the fingerprint is derived from the file's contents, it uniquely identifies that file and serves as the de-facto standard way of naming and referring to samples. A **collision** — two different files producing the same hash — is theoretically possible, and MD5 collisions do exist, but they are uncommon enough that the hash remains a reliable identifier in practice. A terminological note deserves emphasis here: the analyst speaks of a **sample** rather than of *the* malware, because the hash identifies one specific file, and the same malicious program may exist in many slightly different files each with its own hash.

This last point exposes a limitation exploited by attackers. Because the hash changes completely when even a single byte of the file changes, an attacker can bypass hash-based detection simply by modifying a trivial detail of the code, producing a different hash that a hash-blacklist — and possibly an antivirus — will not recognize, even though the malware's actual behavior is unchanged.

Practically, most operating systems provide an `md5` utility as a standard command-line tool, so that hashing a file is a single invocation returning the fingerprint. On Windows, analysts who prefer a graphical interface can use a GUI-wrapped tool such as **HashCalc**, which computes MD5, SHA-1, and other digests for a chosen file at once.

The hash of a sample supports several uses. It can **label** a malware sample unambiguously; it can be **shared** with other analysts to identify the same malware across organizations; and it can be **searched online** to discover whether someone else has already identified and documented the file, sparing duplicated effort.

A more refined variant is the **import-hash** (often called *imphash*). Instead of hashing the whole file, the import-hash is calculated over the set of functions the sample imports from external libraries. Its value lies in clustering: similar samples belonging to the same malware family tend to share the same import-hash, because different versions of the same malware will most likely rely on the same set of APIs. Two files that differ byte-for-byte — and therefore have different MD5 hashes — can thus still be recognized as relatives through a shared import-hash.

Strings

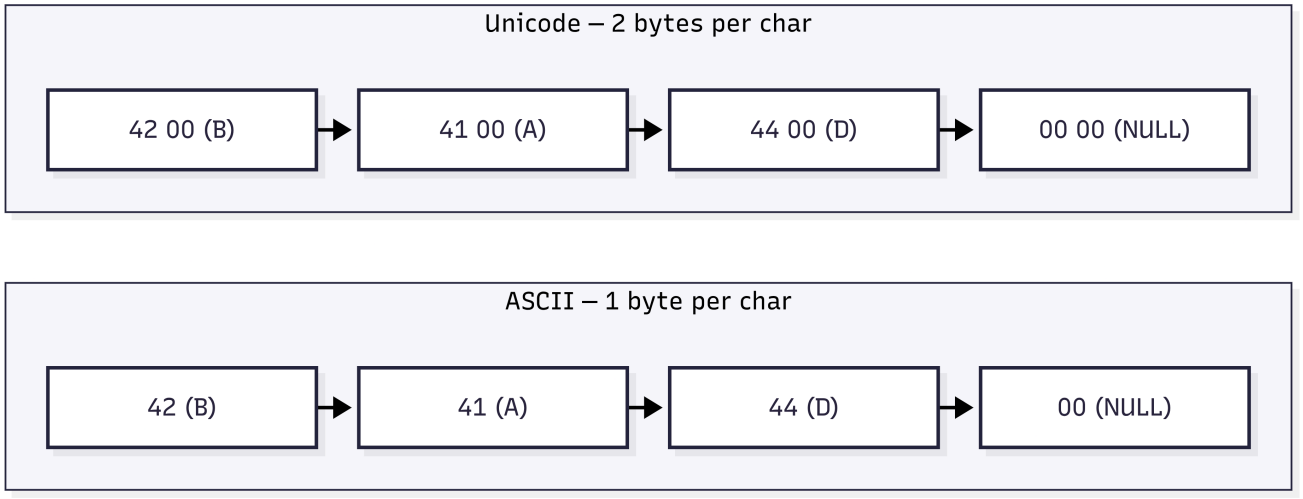
Among the most immediately rewarding artifacts to recover from an executable are its **strings**. A string is any sequence of printable characters embedded in the file, and such sequences frequently leak information the author never intended to expose to a casual observer: file paths, error messages, URLs, IP addresses, registry keys, function names, and fragments of user-interface text. Reading the strings of a sample is therefore a cheap and fast way to form a first hypothesis about what the program does.

To recover strings correctly one must understand how they are stored in memory, because a string is delimited rather than length-prefixed: every string is **terminated by a null value**, written `0x00`, which marks where the sequence of characters ends. The encoding of the individual characters, however, differs between the two conventions an analyst will encounter.

In **ASCII**, each character occupies eight bits — a single byte. The word *BAD*, for example, is stored as the three bytes `42 41 44` (the ASCII codes for B, A, and D) followed immediately by the null terminator `00`. The layout is compact: one byte per character, one terminating byte at the end.

In **Unicode**, each character occupies sixteen bits — two bytes. Microsoft refers to these as **wide characters**. The same word *BAD* now appears as `42 00 41 00 44 00`, each printable byte interleaved with a `00`, and the string closes with a double-byte null terminator `00 00`. Recognizing this interleaving pattern matters in practice, because a tool or an eye tuned only to ASCII may misread wide-

character strings or miss them entirely.



The `strings` Command

The canonical tool for this task is the **`strings`** command. It is native to Linux and OSX and is also available for Windows, and its operation is simple: it scans a file and prints out every string it finds that is three or more characters long. The three-character minimum is a heuristic that filters out much of the incidental noise while still capturing meaningful text.

The catch is that not every printed string is significant. The output mixes genuinely informative strings with **noise** — accidental byte sequences that happen to be printable but carry no meaning. Interpreting the output is therefore an act of judgment. Consider a representative run against a sample, `bp6.ex_`, which prints a list including short meaningless fragments such as `VP3`, `VW3`, `t$@`, `D$4`, `e-@`, and `M}C`, alongside several strings that genuinely tell a story:

- `GetLayout` and `SetLayout` are **Windows functions**, revealing that the program calls into the Windows API.
- `GDI32.DLL` is a **Dynamic Link Library**, the graphics-related library, indicating the program links against it.
- `99.124.22.1` looks like an **IP address**, a strong hint that the program has network functionality and a specific host it may contact.
- `Mail system DLL is invalid.!Send Mail failed to send message.` looks like an **error message**, suggesting the program has email-sending capability and reports failures.

The short, cryptic fragments in **bold** — `VP3`, `VW3`, `t$@`, `D$4`, `e-@`, `M}C` — can safely be ignored as noise, while the recognizable API names, the library name, the IP address, and the error text together sketch a program that

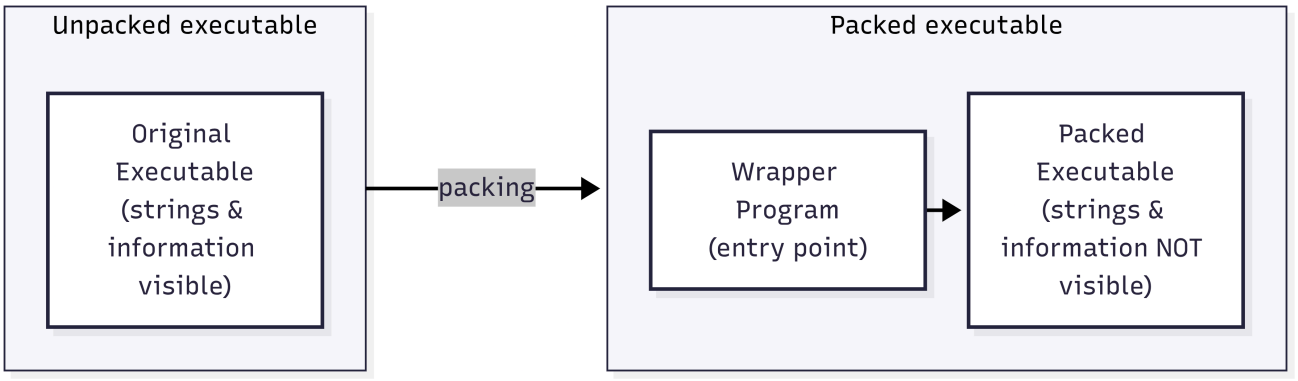
manipulates graphics, communicates over the network, and sends mail. From a handful of recovered strings the analyst has already inferred several concrete capabilities, and all of this without executing a single instruction.

Obfuscation Through Packing

The string-based approach is powerful, which is exactly why malware authors work to defeat it. The principal countermeasure is **packing**. Packing compresses the code of the executable much as a Zip archive compresses a file, and the immediate consequence for the analyst is that most strings and instructions become unreadable, because in their compressed form they appear as essentially random bytes rather than legible text or code.

An important historical nuance is that packing was not invented for malicious purposes. It was originally an obfuscation-and-compression mechanism designed to cope with limited memory and storage, shrinking a program's footprint. Attackers subsequently adopted it because the same compression that saves space also conveniently hides the original code of the malware, concealing the very strings and instructions that basic static analysis relies upon.

Mechanically, a packed executable is restructured into two parts. The original program — with all its strings and other information plainly visible — is compressed and stored as inert data, no longer directly executable in place. Wrapped around it is a small piece of code called the **wrapper** (or unpacker). When the packed file is run, execution begins in the wrapper, which decompresses the original program back into memory and then transfers control to it. The result is that static inspection of the file on disk shows the analyst only the wrapper; the meaningful body of the malware remains hidden inside the packed, unreadable region until runtime.



The practical lesson is that when `strings` returns almost nothing intelligible — only a short wrapper's worth of text and a sea of random-looking bytes — packing

should be suspected. The absence of readable strings is itself an indicator.

Detecting Packers with PEiD

Because packing is so common, tools exist to detect it and, where possible, to name the specific packer used. A classic example is **PEiD**. Given a file, PEiD inspects characteristic markers and reports information such as the **entry point**, the **EP section** (the section containing the entry point), the **file offset**, the **first bytes** at the entry point, the **linker version**, and the **subsystem**. Its most useful output is a line identifying the packer signature — for instance recognizing that a sample was packed with a particular version range of **UPX**, a widely used packer. A telltale sign visible even without PEiD's signature database is an entry-point section bearing a packer's own name, such as a section labeled `UPX1`, since packers frequently rename the sections they create.

PEiD carries a serious warning of its own. Some of its plugins may **silently run the malware** in the course of trying to identify the packer. This means that merely attempting to detect the packer can inadvertently execute the sample, with all the danger that entails. The consequence is a strict operational rule: the testing virtual machine must be properly isolated before such tools are used, so that an accidental execution is contained.

File Execution in Windows

To interpret the headers that basic static analysis inspects, one must first understand what happens when an executable is launched. When a user double-clicks an executable file in Windows, the operating system — specifically the component known as the **loader** — examines the file in order to understand what to do with it. The loader must locate and correctly interpret everything the program needs to run.

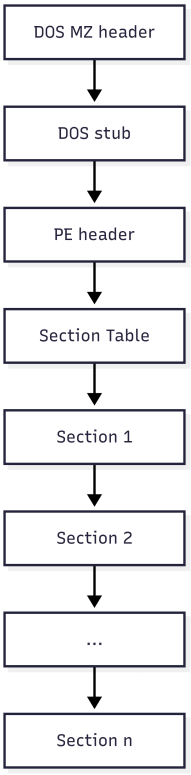
An executable file is not a flat blob; it contains several distinct kinds of content. It holds **code**, the actual machine instructions to be executed; **data**, the values the program operates on; **other resources** such as icons, menus, and embedded media; and **metadata**, descriptive information about the file itself. For the loader to make sense of all this, the information must be organized according to a well-defined structure that the loader knows how to read. That structure, on Windows, is the Portable Executable format.

The Portable Executable (PE) File Format

The **Portable Executable (PE)** format is the file format used by Windows executable files, object code, and DLLs alike. Its single greatest convenience is precisely this universality: one file format serves several different usage scenarios, which means that executables (EXEs) and dynamic link libraries (DLLs) are treated in a very similar way despite their different roles. Structurally, the PE format is a data structure that carries all the information Windows needs in order to load the file into memory and execute it. Almost every file executed on Windows is in PE format, which makes mastery of this format central to static analysis.

Overall Layout

A PE file begins with an **MS-DOS header and stub**, retained purely for backward compatibility. Following this legacy preamble come the **PE headers**, which contain the metadata used to load and execute the file correctly, and then a **section table** that describes the sections. Finally the file's actual content is organized into **sections**, each of which contains a specific kind of data — one section for executable code, another for global variables, another for resources such as multimedia, and so on. Each section is described by an entry in the section table, and usually at least two sections are present in any real executable.



Common Section Names

Certain section names recur so often that they function almost as a vocabulary, though their exact spelling depends on the toolchain. The most commonly present sections in an executable are the following. The **executable code section** is named `.text` under Microsoft conventions, or `CODE` under Borland. The **data sections** are named `.data`, `.rdata`, or `.bss` under Microsoft, or `DATA` under Borland; they hold initialized data, read-only data, and uninitialized data respectively. The **resources section** is named `.rsrc`. The **export data section** is named `.edata`, and the **import data section** is named `.idata`; these list, respectively, the functions the file makes available to others and the functions it needs from others. Finally the **debug information section** is named `.debug`.

A crucial caveat accompanies these names: they are actually **irrelevant to the operating system**, which ignores them and treats them as present only for the convenience of the programmer. The true nature of a section is determined by its characteristics flags, not by its name. The practical implication for the analyst is that section names cannot be trusted — one can encounter very different names in the wild, and packers in particular routinely assign their own idiosyncratic names to the sections they generate.

DOS Header and Stub

The **DOS header** occupies the first 64 bytes of the file. It exists solely for the eventuality that the program is launched from within DOS: in that case DOS can recognize the file as a valid executable and run the accompanying **DOS stub**. The stub itself is trivial — it usually does nothing more than print a message such as "This program must be run under Microsoft Windows" and exit, a graceful refusal on legacy systems.

Two fields in this header are essential to the analyst. The first is the **magic signature** at the very beginning of the file, whose value is `5A4D` in hexadecimal, corresponding to the ASCII characters **"MZ"** (the initials of Mark Zbikowski, a Microsoft architect). The presence of `MZ` at offset zero is the first confirmation that a file is a DOS/Windows executable. The second essential field is **`e_lfanew`**, located at offset `0x3C`, which is a pointer holding the offset to the PE header. In other words, `e_lfanew` tells the loader where, further inside the file, the real PE header begins, allowing it to skip over the DOS compatibility material.

A subtle but important detail emerges when inspecting these values in a raw hexadecimal dump: multi-byte numeric values are stored in **little-endian** order, meaning the bytes appear reversed relative to how the number is written. The

`e_lfanew` field pointing to a PE header at offset `0x000000E0`, for instance, appears in the file as the byte sequence `00 00 01 00` — the bytes laid out in reverse order. At the location it points to, one finds the **PE signature** `50 45 00 00` — the ASCII characters "PE" followed by two null bytes — marking the true start of the PE header. Reading such a dump, the analyst can visually confirm the chain: `4D 5A` ("MZ") at the top, the DOS stub text "This program must be run under Windows" in the middle, the `e_lfanew` pointer near offset `0x3C`, and the `PE` signature at the offset that pointer designates.

The Section Table

Once the three parts of the PE header have been parsed, the loader arrives at the **section table**, the directory that describes each section defined in the file. The table consists of a series of entries, one per section, and each entry is exactly **40 bytes** long. The number of entries is precisely the **NumberOfSections** value read earlier from the COFF File Header, so the two structures are read in tandem: the header states the count, and the table provides that many descriptors. Each 40-byte entry is an **IMAGE_SECTION_HEADER**, and its fields together tell the loader everything it needs to place the section correctly both on disk and in memory.

The fields of a section-table entry are the following.

The **Name** field holds the section name without its terminating character — the familiar `.text`, `.data`, `.rsrc`, and so on. As stressed earlier, this name is a convenience only and is not trusted by the loader.

The **VirtualSize** field gives the size of the section once it is loaded in memory: that is, how much space the loader will allocate for it in RAM.

The **VirtualAddress** field gives the **RVA** of the section — the address, relative to the image base, at which the section will reside in memory.

The **SizeOfRawData** field gives the size of the section's data as it exists in the file on disk.

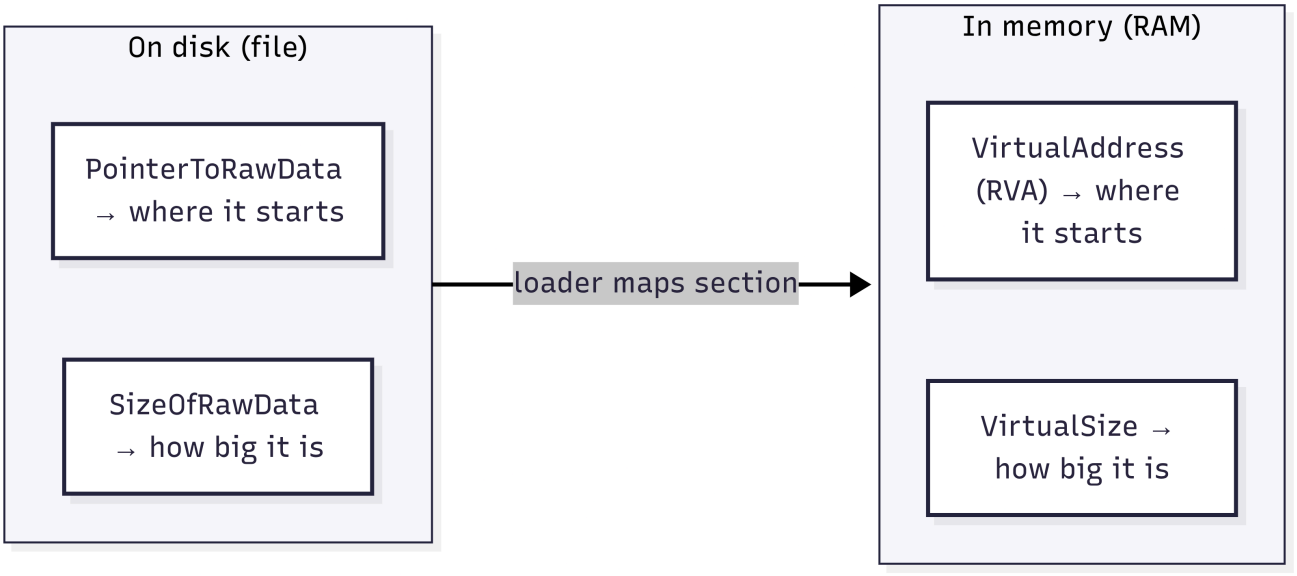
The **PointerToRawData** field gives the offset from the file's beginning to the section's data on disk — where, physically in the file, the section's bytes start.

The fields **PointerToRelocations**, **PointerToLinenumbers**, **NumberOfRelocations**, and **NumberOfLinenumbers** relate to relocation and

line-number information used chiefly in object files, and are of secondary importance for typical executable analysis.

The **Characteristics** field is, as in the file header, the most analytically significant. It contains flags describing the fundamental nature of the section: whether the section contains executable code, whether it contains initialized data, whether it contains uninitialized data, and whether it can be written to or read from. These flags, not the name, are what the loader and the analyst rely upon to know what a section truly is. A section flagged as both writable and executable, for instance, is a classic red flag, since legitimate code sections are normally executable but not writable.

The pairing of the two size fields and the two location fields is the conceptual key to the whole section table. **VirtualSize** with **VirtualAddress** describes the section *in memory*; **SizeOfRawData** with **PointerToRawData** describes the same section *on disk*. Because memory and disk use different alignment rules, these paired values usually differ, and the discrepancy between them is not an error but a designed feature of the format.



PE Sections

After the section headers come the sections themselves — the actual code, data, and resources the descriptors point to. The way these sections are laid out differs between the file on disk and the image in memory, and understanding both layouts is essential.

In the file on disk, each section begins at an offset that is some multiple of the **FileAlignment** value found in the Optional Header. Because a section's real data

rarely fills an exact multiple of that alignment unit, the space between one section's data and the next is filled with `00` **byte padding**. This padding is not meaningful content; it exists only to satisfy the alignment requirement.

When loaded into RAM, the sections are aligned differently: they always start on a **page boundary**, so that the first byte of each section corresponds to the beginning of a memory page. The size of a page depends on the processor architecture — on x86 CPUs pages are **4 kB** aligned, whereas on IA-64 they are **8 kB** aligned. This in-memory alignment value is the **SectionAlignment** field, also found in the Optional Header. The two alignments — FileAlignment on disk and SectionAlignment in memory — are what make the on-disk and in-memory layouts of the same file diverge.

IMAGE_SECTION_HEADER: Virtual Size versus Size of Raw Data

The distinction between the memory description and the disk description of a section is not merely structural bookkeeping; it is a diagnostic tool. Restating the two fields plainly: **VirtualSize** measures the section as it will exist in **RAM**, while **SizeOfRawData** measures the section as it exists on **DISK**.

For a well-behaved executable, these two values are normally equal, or very nearly equal, for the `.text` (code) section — the amount of code on disk matches the amount of code in memory, give or take alignment rounding. A striking departure from this equality is a hallmark of packing. In a **packed executable**, the **VirtualSize** of the `.text` section is much larger than its **SizeOfRawData**. The reason is intuitive once the packing mechanism is recalled: on disk the code sits compressed and therefore small, but at runtime the wrapper decompresses it, expanding it into a much larger region of memory that the section header must reserve in advance. A large virtual size backed by a tiny raw size means the section will "grow" dramatically when executed — exactly what an unpacker does.

A concrete section table for a hypothetical `PackedProgram.exe` makes the pattern vivid. Reading each section's virtual size against its raw size exposes several anomalies:

Name	Virtual Size	Size of Raw Data
`.text`	A000	0000
`.data`	3000	0000

Name	Virtual Size	Size of Raw Data
`.rdata`	4000	0000
`.rsrc`	19000	3400
`Dijfpds`	20000	0000
`.sdfuok`	34000	3313F
`Kijijl`	1000	0200

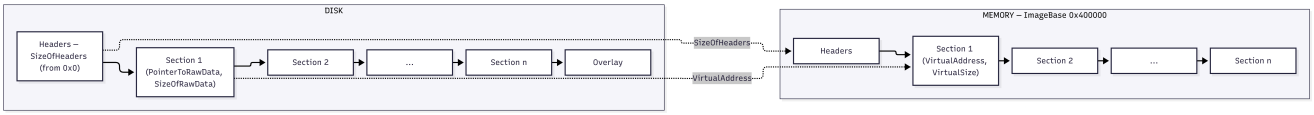
Two categories of suspicion appear at once. First, several standard sections — `.text`, `.data`, `.rdata` — report a nonzero virtual size but a **raw size of zero**, meaning they occupy space in memory yet contribute no data in the file; their contents must be produced at runtime by unpacking. Second, the section names themselves betray the packer: alongside the conventional names sit meaningless, machine-generated names such as `Dijfpds`, `.sdfuok`, and `Kijijl`, which no ordinary compiler would emit. Both signals — the virtual/raw size mismatch and the nonsensical section names — independently point to a packed sample.

Loading in Memory

Bringing the pieces together, one can now trace the complete transformation a PE file undergoes as it moves from disk to memory. On disk, the file begins at offset `0x0` with the **headers**, whose combined length is exactly the **SizeOfHeaders** value. The sections follow the headers on disk, each located by its **PointerToRawData** and extending for its **SizeOfRawData**. At the very end of the file, after the last section, there may be an **overlay** — appended data that lies beyond the region described by the section table and is not mapped into memory by the normal loading process (malware sometimes hides payloads here).

When the loader maps the file into memory, it places the image at the **ImageBase**, the preferred load address — commonly `0x400000`. The headers are copied first, spanning **SizeOfHeaders**. Then each section is placed at its **VirtualAddress** (interpreted relative to the image base) and given room equal to its **VirtualSize**. Because in-memory alignment (SectionAlignment, page-aligned) differs from on-disk alignment (FileAlignment), the sections end up spaced differently in memory than they were in the file: gaps open up between them to satisfy page alignment, and a section whose VirtualSize exceeds its SizeOfRawData is allocated more room than its disk bytes filled, the remainder being zero-initialized.

The relationships among the header fields become concrete in this mapping. **SizeOfHeaders** measures the header block that sits at the top of both layouts. **PointerToRawData** and **SizeOfRawData** locate and size each section *on disk*, while **VirtualAddress** and **VirtualSize** locate and size the same section *in memory*. The overlay at the bottom of the disk file has no counterpart in the mapped image.



The reason the padding and alignment exist is efficiency of memory protection and access: by making each section begin on a page boundary, the operating system can apply page-level permissions — read, write, execute — to each section independently, matching the permission flags stored in the section's **Characteristics** field.

Time Date Stamp

Returning to a field encountered in the COFF File Header, the **TimeDateStamp** deserves a dedicated treatment because of its forensic role. It shows when the executable was **compiled**, and this single fact carries investigative weight: older programs are more likely to be already known to antivirus software, since they have had time to be collected, analyzed, and added to detection databases. A recent compilation date, conversely, may indicate a freshly built sample not yet catalogued.

The stamp must nevertheless be treated with caution, because it is sometimes simply **wrong**. A well-known quirk is that all programs compiled with **Delphi** report the date **June 19, 1992**, regardless of when they were actually built, because of how the Delphi toolchain fills the field. Beyond such artifacts, the date **can also be deliberately faked** by an attacker who edits the field to mislead investigators. The TimeDateStamp is therefore a useful hint but never a proof.

Resource Hacker

The resources bundled into a PE file live in the `.rsrc` section, and a dedicated tool called **Resource Hacker** lets the analyst browse this section directly. Through it one can inspect the **strings, icons, and menus** embedded in the executable. These resources can be revealing: an icon impersonating a legitimate application, menu text in an unexpected language, or embedded strings can all contribute to understanding the sample's disguise and intent.

Clues in Libraries

Some of the most telling evidence about a program's purpose comes not from its own code but from the external functionality it borrows. The PE header **lists the libraries and functions that the program will load**, and the names of those imported functions can reveal a great deal about what the program does. Function names are often descriptive, chosen by the API designers to communicate their effect, and this descriptiveness works to the analyst's advantage. A single imported name can betray a whole capability: the presence of `URLDownloadToFile`, for example, indicates that the program **downloads something** from a URL, immediately suggesting network-retrieval behavior even before any code is examined. Scanning the import list is thus a fast route to a hypothesis about functionality.

Imports

An **import** is a function that a program uses but does not itself contain — a function that is stored in a different program, typically a library. Rather than reimplementing common functionality, a program borrows it from shared libraries, and the mechanism that connects the borrowing executable to the borrowed code is called **linking**. Linking is what binds the external functions to the main EXE so that, at the appropriate moment, a call in the executable actually reaches the code residing in the library.

Linking can be performed in **three ways**: statically, dynamically, or at runtime. Each has different implications for how the sample behaves and for what the analyst will observe in the PE header, and each is examined in turn.

Static Linking

In **static linking**, all the code from the library is **copied directly into the executable** at build time. The library's functions become an integral part of the resulting file, indistinguishable from the program's own code. This approach is **rarely used for Windows executables** but is **more common in Unix and Linux** environments. Its principal consequence is that it makes the executable **large in size**, since it must carry a full copy of every library routine it uses. Its principal advantage is self-containment: static linking is the main way to go if one wants **self-contained code** that depends on no external files to run. For the analyst, static linking is inconvenient, because the library code is fused into the

sample and there is no import entry announcing which library functions are present.

Dynamic Linking

Dynamic linking is the **most common method** on Windows. Here the library code is not copied into the executable; instead, the host operating system **searches for the necessary libraries when the program is loaded** and connects them at that point. The executable carries only references to the functions it needs, and the loader resolves those references against the actual libraries present on the system at load time.

A visible symptom of dynamic linking appears when a required library is absent. If a needed DLL cannot be found, the program fails to start and the system reports an error such as "The program can't start because `MSVCR100D.dll` is missing from your computer. Try reinstalling the program to fix this problem." This message is a direct consequence of the dynamic-linking model: because the library was expected to be supplied by the system rather than embedded, its absence is fatal at load time. For the analyst, dynamic linking is helpful, because the import table plainly lists the libraries and functions the sample depends on.

Runtime Linking

Runtime linking connects to libraries **only when they are actually needed**, rather than when the program starts. It is **unpopular in friendly, legitimate programs**, but it is **common in malware**, especially in packed or obfuscated malware, because it lets the sample avoid declaring its true dependencies up front in the import table — a form of concealment. It is not exclusively malicious, however; it is also used legitimately by software that supports **dynamic loading of plugins**, where the set of libraries to load is not known until runtime. Technically, runtime linking is **most commonly done with the `LoadLibrary` and `GetProcAddress` functions**: `LoadLibrary` loads a named library into the process at runtime, and `GetProcAddress` then retrieves the address of a specific function within that library so it can be called. The analytical significance is that a sample importing little more than these two functions may be hiding its real behavior behind runtime resolution, and the import table alone will not reveal what it ultimately calls.

Dependency Walker

To inspect a sample's dynamically linked functions, analysts use a tool called **Dependency Walker**, which displays the libraries a program depends on and the functions it draws from them. The tool exposes a simple but powerful heuristic: **normal programs tend to have a lot of DLLs**, reflecting their reliance on the rich ecosystem of Windows libraries, whereas **malware often has very few DLLs**, because it deliberately minimizes its declared dependencies — often precisely because it resolves its real functionality at runtime or is packed.

The contrast is illustrated by two versions of the same-named file. The legitimate `SERVICES.EXE` expands in Dependency Walker into a long tree of dependencies — `MSVCRT.DLL`, `RPCRT4.DLL`, `SSPICLI.DLL`, `NTDLL.DLL`, `PROFAPI.DLL`, `CRYPTBASE.DLL`, and a large number of the modular `API-MS-WIN-CORE-*` libraries covering error handling, file access, handles, heap, I/O, the library loader, the local registry, process environment and threads, string handling, synchronization, and more. This dense forest of imports is typical of a genuine system binary. By contrast, a malicious file masquerading as `SERVICES.EX_` shows almost nothing: it imports only `KERNEL32.DLL` and `WS2_32.DLL`. The sparseness is itself the clue — a program presenting itself as an important system service yet declaring only two libraries is behaving unlike any legitimate counterpart, and the presence of `WS2_32.DLL` (a networking library) hints at network activity.

Dependency Walker also distinguishes, for each module, its **imports** and its **exports**. In its interface the imported functions (the parent's imports, marked **PI**) and the exported functions (marked **E**) of a selected library such as `MSVCRT.DLL` are shown separately, each with its ordinal, hint, and function name. Being able to read both lists lets the analyst see not only what a module takes from others but also what it makes available to them.

Common DLLs

Because so much of a program's intent can be read from which libraries it imports, it is worth knowing what the most frequently encountered DLLs provide. A handful of them account for the bulk of ordinary Windows functionality, and recognizing them lets the analyst infer capabilities at a glance.

`Kernel32.dll` is a very common DLL that contains core functionality, including the access and manipulation of memory, files, and hardware. Almost every program uses it.

`Advapi32.dll` provides access to advanced core Windows components such as the Service Manager and the Registry. Its presence suggests the program interacts with services or registry keys.

`User32.dll` contains all the user-interface components — buttons, scroll bars, and the components responsible for controlling and responding to user actions. A program importing it has, or interacts with, a graphical interface.

`Gdi32.dll` contains functions for displaying and manipulating graphics.

`Ntdll.dll` is the interface to the Windows kernel. Executables generally do not import this file directly; it is almost always imported indirectly by way of `Kernel32.dll`. Consequently, if an executable imports `Ntdll.dll` *directly*, it signals that the author intended to use functionality not normally available to ordinary Windows programs — capabilities such as hiding functionality or manipulating processes frequently reach for this low-level interface. A direct import of `Ntdll.dll` is therefore a suspicious sign.

`WSock32.dll` and **`Ws2_32.dll`** are networking DLLs. A program that accesses either of them most likely connects to a network or performs network-related tasks.

`Wininet.dll` contains higher-level networking functions that implement application protocols such as FTP, HTTP, and NTP. Its presence indicates the program speaks these protocols rather than merely opening raw sockets.

Example: Keylogger

The power of import and export analysis is best seen in a worked example. Consider a sample that turns out to be a **keylogger** — a program that secretly records keystrokes. Its behavior can be reconstructed almost entirely from the functions it imports and exports.

The sample **imports `User32.dll` and uses the function `SetWindowsHookEx`**, which is a popular way for keyloggers to receive keyboard inputs; installing a Windows hook lets the program intercept input events system-wide. It then **exports `LowLevelKeyboardProc` and `LowLevelMouseProc`**, the callback functions through which the hooked keyboard and mouse events are delivered, which the malware uses to capture the data and send it elsewhere. Finally, it **uses `RegisterHotKey` to define a special keystroke** — such as Ctrl+Shift+P — that acts as a trigger to **harvest the collected data** on demand. From these

three imported and exported names alone, the complete operating logic of the keylogger emerges: hook the input, receive it through the callbacks, and retrieve the loot with a secret hotkey. No execution and no disassembly were required to reach this understanding.

Example: A Packed Program

The same import-based reasoning also detects packing. A **packed program** presents **very few functions** in its import table, because all the analyst can see is the **unpacker** — the wrapper — while the real program and its true imports remain compressed and hidden until runtime. A representative import listing for a `PackedProgram.exe` bears this out: from `Kernel32.dll` it imports only `GetModuleHandleA`, `LoadLibraryA`, `GetProcAddress`, `ExitProcess`, `VirtualAlloc`, and `VirtualFree`, and from `User32.dll` only `MessageBoxA`. This particular combination is a fingerprint of unpacking behavior. The pairing of **`LoadLibraryA` and `GetProcAddress`** is the runtime-linking mechanism the unpacked code will use to resolve its real dependencies once it is decompressed, while **`VirtualAlloc` and `VirtualFree`** are the memory-management functions used to allocate the region into which the original program will be unpacked and later to release it. A sample whose entire visible import set consists of these load-and-allocate primitives is almost certainly a packed one, showing the analyst nothing but the machinery of its own unpacking.

Exports

Imports and exports are two sides of the linking relationship, and the distinction between them aligns with the distinction between the two kinds of PE module. **DLLs export functions** — they make functions available for other modules to use — while **EXEs import functions**, consuming functionality provided by libraries. **Both exports and imports are listed in the PE header**, so the analyst can read either list from the same structure.

A useful expectation follows from this division: **exports should be rare in EXEs**, since an executable is normally a consumer of functionality rather than a provider of it. When an EXE does export functions, it is anomalous and worth investigating — as the keylogger example showed, an executable exporting callback functions such as `LowLevelKeyboardProc` is doing something unusual. The rule is a guide rather than a law, however, because **reality is sometimes different**: legitimate executables occasionally export functions, so an export in an EXE raises a flag without by itself proving malice.